

```
# -*- coding: utf-8 -*-  
"""M261_batch.ipynb
```

Automatically generated by Colab.

Original file is located at
<https://colab.research.google.com/drive/1kPkgRa5isjP7tFYNVlF992ExLfkWdvVo>

Introduction to Python

Creator of Python is Guido Van Rossum,Dutch programmer who developed Python Programming in 1991, the first version is 0.9.0.

Features of Python:

1. Easy to learn: it is having limited set of keywords,the syntaxes are simple and written in English. We can avoid brackets and instead we use indentation(1 tab space).
 2. Dynamically typed language: there is no requirement of declaring what type of values are beng stored in a varibale. It is decided during the runtime.
 3. Interpreted language: It reads the code line by line, then it will execute.
 4. Large standard library: It has 7+ crores of library function which consists of rich sets of modules,functions,classes and objects,variable, etc.
- built-in functions means the work of the function is already assigned(predefined).
5. High-level language: It is understandable by the humans as well as the computers.
 6. Platform independent: Python programs can be executed in any form of operating systems (i.e Windows,Linux,MacOS,Unbuntu).
 7. Supports multi paradigm: We can execute python program not only in object oriented but also in functional programming and conditional programming also.
 8. Free and open source: It is freely available in the official website and we can use it free of cost.

```
"""  
a= 100  
type(a)      #type(): to check what type of value is stored
```

```
name='JOE'  
type(name)
```

```
x=8.9870  
type(x)
```

```
q=89  
w=67+89  
print(q)  
print(w)
```

```
#structural program  
a= 100  
b=200  
print(a+b)
```

#functions

```
def add(a,b):  
    print(a+b)  
add(78,56)
```

#OOP

```
class A:  
    def __init__(self,a):  
        self.a=a  
    def __add__(self,others):  
        return self.a+others.a  
obj1=A(90)  
obj2=A(34)  
print(obj1+obj2)
```

"""Introduction to Library Functions:

Library functions the predefined function which consits of: Keyword, Operators, Built-in functions.

1. Keywords: These are unique predefined words.

There are 35 keywords in Python.

There are 3 special keyword: True(1), False (0) ,None(empty)

It is used to store values or to do comparison.
"""

#Syntax of Keyword

```
import keyword  
keyword.kwlist
```

len(keyword.kwlist) #len(): it is used to check the lenght of any datatype/function

```
x=True  
print(x)
```

```
b=False  
print(b)
```

```
c=None  
print(c)
```

```
d=and  
print(d)
```

"""Variables:

It is the name given to the memory location where the values are stored, to be used mulitple times.

Syntax: var_name= value

Example: x=90

```
x: var_name  
90: value
```

There are 2 types:

1. Single line variable
2. Multi line variable

Garbage collection: It is an automatic process of handling memory allocation and deallocation ensuring efficient use of memory.

NOTE: No of variable should be equal to no of values.
"""

```

id(10)          #id(): to check the memory address

id('b')

b=10
id(b)

m=6578346389283
id(m)

n='Brindha'
print(n)

id(n)

a=10 #single line variable
b=23
a,b=10,23 #multi line
x,y,z=101,201,301 #multi line variable

print(a)

print(x)

print(z)

x,y,z=101,201,301

x=101
y=201
z=301
a=501
b=None
print(b)

print(z)

a,b,c=1,None,3
print(b)

"""Identifiers:

It is a name which is used to identify the values.

** All variables are identifiers, but all identifiers are not variable.

Rules of Identifiers:

1. It shouldn't be a keyword.
2. It shouldn't start with a number.
3. It shouldn't contain any special character other than underscore '_'
4. It shouldn't contain space in between them or at the beginning.
5. It shouldn't cross the length of 79 characters.
6. It can have alphanumeric,uppercase & lowercase or '_'.
"""

a2=100
NAME='Selva'
name='Brindha'
_ =23

a2

NAME

name

-

"""DATATYPES:

It is a collection of information which will specify the size and type of the data stored in the variable.

There are 2 categories:

1. Single value datatype(SVD) / Individual Datatype: where a single value is stored in a single variable.

    1. Numeric:
        i. integer (int)
        ii. float
        iii.complex
    2. Boolean:
        i. True
        ii. False

    Example:

    x=90--int
    y=2.345--float
    z=2+3j---complex
    n=True---boolean
    m=False---boolean

    2. Multi value datatype (MVD) / Collection datatype: where multiple values are stored in a single variable.
Categories:
    1. string(str)---'Good morning'
    2. list----[1,2,3,4,5]
    3. tuple---(1,2,3,4,5)
    4. set----{1,2,3,4,5}
    5. dictionary(dict)---{'a':10,'b':20}

Single value datatype(SVD)
1. Numeric:

i.integer(int): real numbers without decimal value.

Eg: p=78

range of int: -infinite to +infinite

There are 2 types of values:

1. default value: the initial value which 0 internally it is False

2. non-default value: all other values apart from 0 is non-default value.

to check the default value: bool()

Default value of int is 0 internally it is False.
"""

```

```

p=78
type(p)
bool(p)

d= -890
type(d)

bool(d)

s=0
bool(s)

"""ii. Float: real number with decimal point.

Eg: f=2.4567

range of float: -infinite to +infinite.

default value of float: 0.0/ False

iii. Complex: combination of real and imaginary value.

a+bj/ a+bJ

'j' is dependent on the value of 'b'

default value of complex: 0j
"""

f=2.34567
type(f)
id(f)

c=2+3j
c

v=4+5J
v

m=2+3K
m

s=1+0j
s

"""2. Boolean: it consists of 2 values which is True(1) and False(0).Both are special keywords.

Use of boolean values:
1. to assign it as a value format.
2. as a resultant while comparison between 2 datatypes.
"""

a=True
type(a)

b=False
type(b)

10>10.0

10==10.0

"""II. Multi Value Datatype(MVD):

i. string(str): collection of characters which is enclosed in single/double/triple quotes.

Syntax: var_name='characters'

In string we can store the characters like lowercase/uppercase alphabets, numbers(0-9), special symbols(!@#$$%^&*_-+=.....)

String is immutable collection.
"""

n='Amrutha'
n

m="Selva  "
len(m)

s="Happy Mother's Day"
len(s)

'''triple quotes: it is used to write multiple lines of string values.
it is used to comment multiple lines.'''
#\n : it indicates new line.
b="'''Good Morning.
How are you?
Are you going to office?
No,i am not feeling well today'''

len(b)

x='90'
type(x)

"""Indexing: it is a subaddress given to each and every block of memory present in a collection.

2 ways:

i. positive indexing: traverse from left to right.

(0,len(collection)-1)

ii. negative indexing: traverse from right to left.

(-1,-len(collection))

**Negative indexing should be used when the length is unknown.

Syntax: var_name=[index_position]

Useing indexing we can classify if a datatype/collection is mutable or immutable.

Mutable: a collection which can be modified.
Immutable: a collection which cannot be modified.

to modify any collection:

Syntax: var_name[index_position]=new_value
"""

k='Programming'
len(k)

```

```

k[3]

k[2]='O'

id(k)

k='PrOgramming'
id(k)

"""2. List: it is used to store multiple elements in a single variable using square brackets [].

Syntax: var_name=[val1,val2,....valn]

list is mutable datatype, which means we can add,delete or replace values.

list is ordered.

list stores duplicate values.

list elements can be accessed directly using indexing.

list stores both homogeneous and heterogeneous values.

homogeneous : same datatypes

heterogenous: diff. datatypes
"""

#homogeneous collection
l=[1,2,3,4,5,2,3,4,5,3,4,5,4,5,5,5,1]
type(l)

l

id(l)

#heterogenous collection
l2=[12,2.34,3+5j,'star',[1,2,3]]
l2

id(l2)

id(l2[0])

l2[0]=1234
l2

"""Functions of list:

#add to elements:

1. append(): used to add one value at a time at the end the list.
Syntax: var_name.append(value)

2. extend(): used to add multiple values at the end of the list at a go.
Syntax: var_name.extend([values])

3. insert(): add value in a given index position.
Syntax: var_name.insert(index_position,value)

#remove elements:

1. pop(): remove values from a list collection either from the given index or from the end the list.
Syntax: var_name.pop(): remove the last element

var.pop(index_position): remove the given index positioned value.

2. remove(): used to delete one element at a time from a list,the given element is removed.
Syntax: var_name.remove(value)
"""

#append()

q=[10,20,30,40,50]

q.append(60)

q

#extend()
q.extend([70,80,90,100])
q

insert()
q.insert(5,55)
q

#pop()
q.pop()

q

q.pop(5)

q

#remove()
q.remove(60)

q

"""3.Tuple: collection datatype, which is used to store homogeneous and heterogeneous values.

we can store duplicate values.

we store the values in tuple in single brackets.

Syntax: var_name=(val1,val2,...,valn)

or

var_name=val1,val2,...valn

the elements in tuple are ordered.

tuple is immutable datatype.

```

default value of tuple (); which is internally False.

```
"""
```

```
t=(1,2,3,4,5,6)
type(t)
```

```
len(t)
```

```
p=10,20,30,40
type(p)
```

```
k=100,
type(k)
```

```
t
```

```
t[0]
```

```
t[4]
```

```
t[3]=14
```

```
t=(12,2.3,4+5j,'star',[1,2,3])
```

```
t
```

```
"""Difference between list and tuple:
```

```
List:
```

```
mutable.
time consuming and is slower than tuple.
consumes more memory.
error prone.
it is used for iteration and deletion of operation.
```

```
Tuple:
```

```
immutable.
faster than list.
consumes less memory.
safer operation.
used for read only operation.
"""
```

```
l=[1,2,3,4,5,6,7,8,9]
```

```
l[4]=14
```

```
l
```

```
"""4. Set: collection datatype where we store unique values.
```

```
it is unordered.
```

```
no duplicate values.
```

```
values in set should be immutable.
```

```
indexing is not possible in set.
```

```
set can store homogeneous and heterogeneous values.
```

```
default value of set is set().
```

```
Syntax: var_name={val1,val2,.....,valn}
```

```
set is mutable.we can add and remove values from set using built-in functions.
```

```
Functions:
```

```
i. add(): it is used to add values/elements in a set.
```

```
Syntax: var_name.add(value)
```

```
ii. pop():it is used to remove elements from the front of a set collection.
```

```
Syntax:var_name.pop()
```

```
iii. remove(): it is used to remove given elements from the set.
```

```
Syntax: var_name.remove(value)
```

```
s={12,23,45,67,89}
type(s)
<class 'set'>
s
{67, 23, 89, 12, 45}
m={1,2.3,True,5+9j,'star',[1,2,3]}
Traceback (most recent call last):
  File "<pyshell#4>", line 1, in <module>
    m={1,2.3,True,5+9j,'star',[1,2,3]}
TypeError: cannot use 'list' as a set element (unhashable type: 'list')
m={1,2.3,True,5+9j,'star'}
m
{(5+9j), 1, 2.3, 'star'}
m
{(5+9j), 1, 2.3, 'star'}
m[0]
Traceback (most recent call last):
  File "<pyshell#8>", line 1, in <module>
    m[0]
TypeError: 'set' object is not subscriptable
s={12,23,45,67,89}
s
{67, 23, 89, 12, 45}
s.add(90)
s
{67, 23, 89, 90, 12, 45}
s.add(123)
s
{67, 23, 89, 90, 123, 12, 45}
s.add(12)
s
{67, 23, 89, 90, 123, 12, 45}
s.pop()
67

s.pop()
23
s
{89, 90, 123, 12, 45}
```

```

s.remove(123)
s
{89, 90, 12, 45}
"""

s={12,23,45,67,89}
type(s)

s.add(90)

s

"""5. Dictionary: it is a collection datatype.
in dictionary we can store key-value pair which is called as items.

we store the values of dictionary in {}.

Syntax: var_name={key:value,...,key_n:value_n}

keys should be unique.

indexing can be done by using key.
"""

d={'a':10,'b':20,'c':[1,2,3],'d':'dog'}
type(d)

len(d)

d

d['c']

d['c']=(1,2,3)

d

d['b']='python'

d

d['b'][3]

d['a']

d['c'][2]

s={11,22,33,11,44,22,55,33,35,56,90}
print(s)
len(s)

"""Slicing:

It is a phenomenon of extracting group of values from a collection sequentially.

Syntax:

var_name[SI:EI+-1:update/steps]

Simplified Syntaxes:

if SI==0 or SI==-1:

var_name[:EI+-1:steps]

if EI== len(collection):
var_name[SI::steps]

if steps==1:

var_name[SI:EI+-1]

reverse of a whole string: var_name[::-1]
"""

x='Dynamically'

# 'Dynamic'

x[0:6+1:1]

# 'ally'
x[7:10+1:1]

s='Programming'

# 'gram'

s[3:7:1]

v='Polymorphism'

# 'morph'
v[4:9:1]

# 'Poly'
v[0:4:1]

m='Python'

# 'thon'

m[2:5+1:1]

# 'noht'
m[-1:-5:-1]

k='language'

# 'age'
k[5:7+1:1]
# 'lang'
k[0:4:1]

# 'ega'
k[-1:-4:-1]

l='inheritance'

# 'inherit'--- SI==0
l[:6+1:1]

# 'tance' ----EI==len(collection)

```

```

l[6::-1]

# 'herit' ---- steps==1
l[2:6+1]

# 'ecnatirehni'

l[::-1]

# slicing on list collection

list1=[10,22,35,89,110,78,45]
list1[2:5]

list1[::-1]

# slicing on tuple collection

t=(90,100,111,567,345)
t[1:3]

# in dictionary collection we can apply slicing concept only on values.
d={'a':10,'b':20,'c':[1,2,3,4,5],'d':'program'}

d['c'][1:3]

d['d'][:2+1]

"""#Practise Questions

* Given `s = 'PythonProgramming'`, extract `Python`.

* Extract `Programming` from the same string.

* Extract the first 6 characters of `s`.

* Extract the last 5 characters of `s`.

* Extract characters from index 3 to 9.

* Reverse the string `s` using slicing.

* Extract every second character from `s`.

* Extract every third character from `s`.

* Extract the string in reverse order, skipping one character each time.

* From `s = 'DataScience'`, extract `Science` using negative indexing.


* Given `nums = [10, 20, 30, 40, 50, 60, 70, 80]`, extract the first 4 elements.

* Extract the last 3 elements.

* Extract elements from index 2 to 5.

* Extract all elements except the first and last.

* Extract the middle 4 elements.

* Reverse the list using slicing.

* Extract every alternate element.

* Extract elements at odd indices only.

* Extract elements at even indices only.

* From `nums`, create a new list containing elements in reverse order, skipping every second element.


* Given `t = ('apple', 'banana', 'cherry', 'date', 'fig', 'grape')`, extract the first 3 items.

* Extract the last 2 items.

* Extract items from index 1 to 4.

* Extract all items except `banana` and `date` using slicing and concatenation.

* Extract `('cherry', 'date', 'fig')`.


* Reverse the tuple using slicing.

* Extract every second item.

* Extract every third item starting from index 1.

* Extract the tuple in reverse order, skipping one item each time.

* Extract the first 4 items in reverse order.

* Given `s = 'MachineLearning'`, extract `Learning` and reverse it.

* Given `lst = [1,2,3,4,5,6,7,8,9,10]`, extract all odd numbers using slicing.

* Given `t = (10,20,30,40,50,60,70,80)`, extract `(80, 60, 40, 20)` using slicing.

* Given `s = 'ArtificialIntelligence'`, extract every second character in reverse order.
"""

# From `s = 'DataScience'`, extract `Science` using negative indexing.
s='DataScience'
s[-7:]

# Given `nums = [10, 20, 30, 40, 50, 60, 70, 80]`, extract the first 4 elements.

nums = [10, 20, 30, 40, 50, 60, 70, 80]
nums[0:4]

# Given `t = ('apple', 'banana', 'cherry', 'date', 'fig', 'grape')`, extract the first 3 items.

t = ('apple', 'banana', 'cherry', 'date', 'fig', 'grape')
t[3::-1]

"""#TypeCasting/ Type Conversion

It is a process of converting from one datatype to another.

Syntax:

dest_var=datatype(source_var)

```

```
n=123---float
m=float(n)
m=123.0
```

```
int,float,complex,bool,str,list,tuple,set,dict
```

Conversions:

#Single Value Datatype

i. int: float,complex,bool,str (consumes more memory)

ii. float: int(data loss),complex,bool,str(consumes more memory)

iii. complex: bool, str(consumes more memory)

iv. bool: int,float,complex,str(consumes more memory)

#Multi value datatype

v.str: int,float,complex (only if it is in it's original format), list,tuple,set

vi. list: str(consumes more memory),tuple,set,dict(only if the length of each element is 2).

vii. tuple: str(consumes more memory),list,set,dict(only if the length of each element is 2).

viii. set:str(consumes more memory),list,tuple,dict(only if the length of each element is 2).

ix. dict: str(consumes more memory), list,tuple,set.

"""

```
#int
n=123
```

```
type(n)
float(n)
complex(n)
bool(n)
str(n)
```

#Float

```
f=12.345
type(f)
int(f)
complex(f)
bool(f)
str(f)
```

#Complex

```
c=12+34j
type(c)
bool(c)
str(c)
```

#Boolean

```
a=True
b=False
```

```
int(a)
int(b)
```

```
float(a)
float(b)
```

```
complex(a)
complex(b)
```

```
str(a)
str(b)
```

#string (str)

```
p='345'
type(p)
int(p)
float(p)
complex(p)
```

```
g='12.789'
float(g)
complex(g)
```

```
x='python'
type(x)
bool(x)
list(x)
tuple(x)
set(x)
y='collection'
set(y)
```

#dict(x)

#List

```
l=[12,23,45,56,67,89,12,23,34]
type(l)
str(l)
tuple(l)
set(l)
#dict(l)
```

```
l2={(1,2),'pq',[3,4]}
dict(l2)
```

#Tuple

```
t=(12,23,45,56,67,89,12,23,34)
type(t)
str(t)
list(t)
set(t)
#dict(t)
```

```
t2={(1,2),'pq',[3,4]}
dict(t2)
```

#Set

```
s={12,23,45,56,67,89,34}
```



```

str(s)
list(s)
tuple(s)
#dict(s)

s2={(1,2), 'pq'}
dict(s2)

#Dictionary

d={'a':100, 'g': 'general', 'c':12.34, 'd': 'declare', 'e': 'declare'}
type(d)
str(d)
list(d)
list(d.values())
list(d.items())

tuple(d)
tuple(d.values())
tuple(d.items())

set(d)
set(d.values())
set(d.items())

"""#Copy Operation

It is a process of copying the content from one variable to another keeping the original content intact.

There are 2 types:

General/Normal copy
1. Shallow copy
2. Deep copy

int,float,complex we cannot apply copy operation cz it is immutable.

str,tuple--immutable

set--no duplicate

dict-- key-value is to be stored

Only compatible datatype is list[mutable datatype],store duplicate values.

#General/Normal Copy

The content of the variable will be copied in the same memory location.

#Shallow Copy

The content of the variable is copied in different memory location.

Syntax:

import copy
copy.copy(source_var)

or

dest_var=source_var.copy()

In nested collection when shallow copy is done the nested values gets copied in the same memory location which when modified can effect the original collection.

#Deep Copy

It copies the content of the original variable into another variable but in different memory location, even if it is a nested collection.

Syntax:

import copy
dest_var=copy.deepcopy(source_var)
"""

```

There are 2 types:

General/Normal copy
 1. Shallow copy
 2. Deep copy

int,float,complex we cannot apply copy operation cz it is immutable.

str,tuple--immutable

set--no duplicate

dict-- key-value is to be stored

Only compatible datatype is list[mutable datatype],store duplicate values.

#General/Normal Copy

The content of the variable will be copied in the same memory location.

#Shallow Copy

The content of the variable is copied in different memory location.

Syntax:

```

import copy
copy.copy(source_var)

```

or

```

dest_var=source_var.copy()

```

In nested collection when shallow copy is done the nested values gets copied in the same memory location which when modified can effect the original collection.

#Deep Copy

It copies the content of the original variable into another variable but in different memory location, even if it is a nested collection.

Syntax:

```

import copy
dest_var=copy.deepcopy(source_var)
"""

```

```

n=[10,20,30,40]

```

```

#general copy
m=n

```

```

m

```

```

id(n)

```

```

id(m)

```

```

m[1]=123

```

```

m

```

```

n

```

```

x=[1,2,3,4,5,6]
id(x)

```

```

y=x.copy()
y

```

```

id(y)

```

```

y[2]=890
y

```

```

x

```

```

m=[10,20,30,40,[1,2,3,4,5]]

```

#nested collection is a collection inside another collection

```

id(m)

```

```

id(m[4])

```

```

n=m.copy()
n

```

```

id(n)

```

```

id(n[4])

```

```

n[4][3]=400
n

```

```

m
n[3]=999
n
m

#general copy
p=[12,90,111,23,456]
id(p)

#dest_var=source_var syntax of general copy

q=p
q

id(q)

q[3]=45678
q

p

#shallow copy
x=[1,2,3,4,5,6,7]
id(x)

#dest_var=soource_var.copy() syntax of shallow copy

y=x.copy()
y

id(y)

y[4]=500
y

x

m=[100,112,123,334,555,[10,20,30,40,50]]
id(m)

#memory location of 555
id(m[4])

#memory location of [10,20,30,40,50]
id(m[5])

#make shallow copy of m variable

n=m.copy()
n

id(n)

id(n[4])

id(n[5])

n[4]=55
n

m

n[5][2]=9000
n

m

#deep copy

k=[90,100,110,120,130,140,[12,23,34,45]]

import copy
w=copy.deepcopy(k)

w

id(k)

id(w)

id(k[6])

id(w[6])

w[6][0]=1200
w

k

"""#Operators

The library function which are meant to perform some specific task or operation.

operator: symbols
operand: values on which the operators are applied.

Eg: 10+20

10,20: operands
+ : operator

Types:

1. Arithmetic (+,-,/,*,...)
2. Logical (AND,OR,NOT)
3. Relational (<,>,....)
4. Bitwise (&|,>>,<<,.....)
5. Assignment (=)
6. Membership (in,not in)
7. Identity (is, is not)

1. Arithmetic Operator: are used to perform mathematical calculations

i. Addition(+): op1+op2 [all SVD,str,list,tuple supports]
ii. Substraction(-): op1-op2 [all SVD,set supports]
iii. Multiplication(*): op1*op2 [SVD]
                        op*n [MVD] n:no of times the value should be multiplied.value of n should be always integer.
[all SVD, string,list, tuple supports]
iv.Division:
    . True Division (/): gives all the values including the float number.

```

```
Eg: 10/3=3.3333333333333335
. Floor Division(//): gives only the real number.
Eg: 10//3=3
. Modulus(%): gives the remainder number.
Eg: 10%3=1
only SVD supports division
```

```
v. Power(**): op**n
both the operand and the 'n' value should be SVD
Only SVD supports power.
"""
```

```
#Addition
```

```
n=100
m=123
add=n+m
print(add)
```

```
#we concatenate MVD
```

```
l1=[1,2,3,4,5]
l2=[10,20,30,40,50]
print(l1+l2)
```

```
k='hi'
l='hello'
print(k+l)
```

```
89.23+23.45
```

```
(12+2j)+(23+9j)
```

```
True+False
```

```
(1,2,3,4)+(4,5,6,7)
```

```
{1,2,3,4,5}+(2,3,4,5,6,7,8,9)
```

```
{'a':10,'b':20}+{'c':30,'d':40}
```

```
100-30
```

```
12.34-9.89
```

```
(12+2j)-(23+9j)
```

```
True-False
```

```
'good'-'morning'
```

```
l1=[1,2,3,4,5]
l2=[10,20,30,40,50]
print(l1-l2)
```

```
(1,2,3,4)-(4,5,6,7)
```

```
{1,2,3,4,5,100}-{2,3,4,5,6,7,8,9}
```

```
{'a':10,'b':20}-{ 'c':30,'d':40}
```

```
12*89
```

```
12.34*78
```

```
(1+4j)*(2+6j)
```

```
True*False
```

```
'python'*2
```

```
[10,203,44,555,789]*3
```

```
{11,22,33,44}*2
```

```
{'a':10,'b':20}*2
```

```
"""2. Logical operator: it will perform basic logical operations between 2 values/datatypes.
```

```
There are 3 types:
```

1. Logical AND (and): op1 and op2
2. Logical OR (or): op1 or op2
3. Logical NOT(not): not(op)

```
Assignment Operator: it is used to assign values in a variable
```

```
'=' is the assignment operator
```

```
Eg: b=12.34
```

```
Membership Operator: it will check if a value is present in the collection.
```

```
in: val in collection
```

```
Eg: 10 in [10,20,30,40,50]
```

```
True
```

```
Eg: 70 in [10,20,30,40,50]
```

```
False
```

```
not in: val not in collection
```

```
Eg:
10 not in [10,20,30,40,50]
False
```

```
100 not in [10,20,30,40,50]
True
"""
```

```
10 in [10,20,30,40,50]
```

```
100 in [10,20,30,40,50]
```

```
10 not in [10,20,30,40,50]
```

```
100 not in [10,20,30,40,50]
```

```
d={'a':10,'b':20,'c':30}
```

```
10 in d.values()
```

```
h={'z':'zebra','c':'cat','l':'lion'}
'zebra' in h.values()
```

```
"""Identity Operator: it check if 2 or more variable is pointing to the same memory location.
```

```
is: var1 is var2
```

```
is not: var1 is not var 2
"""
```

```
a=100
b=123
c=234
d=123
```

```
id(a)
```

```
id(b)
```

```
id(c)
```

```
id(d)
```

```
a is b
```

```
a is d
```

```
b is d
```

```
a is not b
```

```
b is not d
```

```
"""#Relational Operator
```

```
It is used to compare between 2 or more operands and the results will be in True or False.
```

```
1. Double Equal to (==): op1 == op2
```

```
SVD compare the values
```

```
MVD it compares type and values.
"""
```

```
#Double equal to(==)
7==8
```

```
9==9.0
```

```
[10,20,30,40]==(10,20,30,40)
```

```
[10,20,30,40]==[10,20,30,40]
```

```
[10,20,30,40]==[10,20,30,40,50]
```

```
(10,20,30,40)==(10,20,30,40)
```

```
{10,20,30,40}=={10,20,30,40}
```

```
{'a':10,'b':20}=={'a':10,'b':20}
```

```
#in string it will check with ASCII values.
```

```
'hello'=='Hello'
```

```
#Not equal to (!=)
```

```
10!=20
```

```
'hello'!='hi'
```

```
[10,20,30,40]!=(10,20,30,40)
```

```
#Greater than (>): op1>op2 [SVD]
```

```
23>12
```

```
23>34
```

```
"""#Greater than operator
collection 1>collection 2
```

```
[val1,val2,...,valn]>[v1,v2,...,vn]
if
val1>v1---True
val1<v1---False
val1= v1---it will check the next values
```

```
except complex and dict all other datatypes support greater than operator
"""
```

```
#greater than operator for MVD
```

```
[12,34,56]>[12,46,59]
```

```
[12,23,45,67]>[12,23,34,45]
```

```
(10,20,30,40)>(20,30,40,50)
```

```
(102,103,105,107,109)>(102,103,105)
```

```
#in set collection it check if the values from the second set is
```

```
#present in the first one
```

```
{12,23,34,45}>{12,34}
```

```
{'a':10,'b':20}>{'a':100,'b':200}
```

```
(12+3j)>(2+3j)
```

```
"""#Lesser than operator(<):
```

```
SVD: op1<op2
```

```
MVD: collection1<collection2
```

```
[val1,val2,...,valn]<[v1,v2,.....,vn]
```

```
if
val1<v1--- True
val1>v1---- False
val1 == v1---check the next values
```

```
string collection it will compare with ASCII values.
```

```
set collection:it will check if the values from the first set collection is present on second set collection.
"""
```

```
10<20
```

```
2.345<2.333
```

```
[10,20,30,40]<[20,30,40,50]
```

```
[10,20,30,40]<[10,20,30,40,50]
```

```
'hi'<'Hi'
```

```
{12,23,34,45}<{12,34}
```

```
"""#greater than equal to (>=):
```

```
SVD:op1>=op2
```

```
MVD: collection 1>= collection 2
```

```
[val1,val2,....,valn]>=[v1,v2,....,vn]
```

```
if
val>v1---True
vall<v1 ----False
vall==v1---check set of values,if all the values are same still the output will be True.
"""
```

```
[10,20,30,40]>=[10,20,30,40]
```

```
"""#Lesser than equal to(<=):
```

```
#Bitwise Operator:
```

```
It will be performed on the binary representation of integers.
```

1. Bitwise AND (&): op1 & op2
2. Bitwise OR (|): op1 | op2
3. Bitwise XOR(^): op1 ^op2
4. Bitwise NOT(~): ~(op)=-(op+1)
5. Bitwise leftshift(<<): op<< n [n is for the no of shifts of the value]
6. Bitwise rightshift(>>): op>> n [n is for the no of shifts of the value]

```
Difference between Logical and Bitwise operator:
```

```
Logical:
```

```
any datatype used.
the internal value of the given operand is considered.
keywords are used.
```

```
Eg: and,or,not
```

```
Bitwise:
```

```
only integer values used.
integer values are converted to binary numbers.
special symbols are used.
```

```
Eg: &,|,~,<<,>>,<^
"""
```

```
11 & 18
```

```
3 & 8
```

```
23 | 12
```

```
#bitwise not
```

```
-(op+1)
```

```
(-9)=-(-9+1)=-(-8)=8
```

```
~(102)
```

```
-(102+1)=-103
```

```
~(-9)
```

```
~(102)
```

```
"""#More Practise question on slicing
```

1. String
s = 'ComputerVision'
Extract 'Vision' using negative indexing.
2. String
s = 'OpenAIChatGPT'
Extract every 4th character starting from the first character.
3. String
s = 'CyberSecurity'
Create a new string by removing the first and last three characters using slicing only.
4. String
s = 'DataVisualization'
Extract the characters from the middle of the string to the end.
5. String
s = 'Knowledge'
Create a new string that contains all characters except every third character.
6. List
marks = [45,56,67,78,89,91,82,73,64]
Extract the first five marks in reverse order.
7. List
languages = ['Python','Java','C','C++','JavaScript','Go','Rust']
Extract only the programming languages that appear after 'C'.
8. List
nums=[3,6,9,12,15,18,21,24,27,30]
Create a list containing every third element starting from 9.
9. List
animals=['Cat','Dog','Lion','Tiger','Elephant','Horse','Zebra']
Extract all elements except the middle one(s) using slicing and concatenation.
10. List
values=[100,200,300,400,500,600,700]
Swap only the first two and last two elements using slicing.
11. Tuple
t=(11,22,33,44,55,66,77,88)
Extract all elements except the first three and last two.
12. Tuple
months=('Jan','Feb','Mar','Apr','May','Jun','Jul','Aug')
Extract the summer months (Apr to Jun) using slicing.
13. Tuple

```

letters=('P','Y','T','H','O','N')
Create the tuple ('N','O','H') using one slicing operation.
14. Nested List
matrix=[[2,4,6],[8,10,12],[14,16,18],[20,22,24],[26,28,30]]
Extract only the first three rows.
15. Nested List
records=[['A',80],['B',75],['C',90],['D',88],['E',95]]
Extract the last four records using slicing.
16. Bytes
b=b'Artificial'
Extract the first six bytes.
17. Bytearray
ba=bytearray(b'ProgrammingLanguage')
Extract 'Language' using slicing.
18. Mixed
data=['Python','Java','C++','SQL','HTML','CSS','JavaScript','Django']
Using slicing only:
- Extract the first four technologies.
- Extract the last three technologies.
- Reverse the entire list.
- Create a new list containing only the middle four technologies.

```

#Input and Output Statements

input statements

input(): built in function usnig this user input value will be taken.
by default the user input value will be in string format.

int(): built in function used to typecast the values in integer.

```
var_name=int(input('Enter the message.'))
```

eval(): built in function to typecaste any datatype during runtime.

```
var_name=eval(input('Enter message.'))
```

output statements:

print(): built function which is used to show/display the output in the terminal/ consol.
"""

```

n=input('Enter value: ')
print(n)
type(n)

```

```

number=int(input('Enter a integer value: '))
print(number)
type(number)

```

```

number=float(input('Enter a integer value: '))
print(number)
type(number)

```

```

data=eval(input("Enter a collection: "))
print(data)
type(data)

```

""">#Control Statement

It is to control the flow of the execution of the program.

types:

1. Conditional/ decisional statement
 - i. simple if
 - ii. if-else
 - iii. elif (else if)
 - iv. nested if
2. Looping statement
 - i. while loop
 - ii.for loop

#Conditional/ Decisional statement:
#simple if

Syntax:

```

if condition:
    #true_statement_block

```

#if-else:

Syntax:

```

if condition:
    #true_statement_block
else:
    #false_statement_block
"""

```

#simple if syntax:

```

if condition:
    #true_statement_block

```

#WAP to check if an integer number is even.

#even no: a number which is divisible by 2.

```

num =int(input('Enter an integer : '))
if num%2==0:
    print('It is an Even number.')

```

#syntax for if-else:

```

if condition:
    #true_statement_block
else:
    #false_statement_block

```

#WAP to check if an integer number is even or odd.

```

num =int(input('Enter an integer : '))
if num%2==0:
    print('It is an Even number.')
else:
    print('It is an Odd number.')

```

#WAP to check if a given string is having 6 characters in it or not.

```

string=input('Enter a string: ')
if len(string)==6:
    print('The string is having 6 characters in it.')
else:

```

```

    print('The string is not having 6 characters in it.')

#WAP to print the square of a number only if it is multiple of 3.

n=int(input('Enter a integer value: '))
if n%3==0:
    print('It is multiple of 3 and the square is',n**2)
else:
    print('It is not multiple of 3.')

#WAP to check if the string is Palindrome or not.

#palindrome: when reversed the value will be same as the input.
'''
mom---mom [Palindrome]

Mom---moM [Not Palindrome]

reverse string: [::-1]
'''

s=input('Enter a string: ')
if s==s[::-1]:
    print('Palindrome string')
else:
    print('Not a Palindrome string')

s=input('Enter a string: ')
rev=s[::-1]
if s==rev:
    print('Palindrome string')
else:
    print('Not a Palindrome string')

#WAP to check if a character is vowel or consonant.
#lower(): built in function which converts uppercase character to lowercase.

char=input('Enter a character to check:').lower()
if char in 'AEIOUaeiou':
    print('Character is Vowel')
else:
    print('Character is Consonant.')

""">#Practice Question on if-else:

1. WAP to check if a given integer is greater than 200.

2. WAP to check if a given integer is multiple of 5 and divisible by 3.

3. WAP to check if a given integer is 2-digit number or not.

4. WAP to print the cube of the number only if the number is divisible by 7.

5. WAP to check if a given character is in uppercase or lowercase.

6. WAP to check if the given input if float or not.

7. WAP to check if the given data is SVD or MVD.

8. WAP to check if the given number is positive or negative number.

9. WAP and consider a tuple collection which consist of 2 values and check if the collection id homogeneous or heterogeneous collection.

#elif statement (else if statement)

it is used when there are multiple conditions to be checked.

Syntax:

if condition1:
    #TSB1
elif condition2:
    #TSB2
.
.
.
elif condition_n:
    #TSB_n
else:
    #FSB
"""

#WAP to find the relation between 2 integer values.

#f-string: it converts any variable to string.

n1=int(input('Enter an integer value for n1: '))
n2=int(input('Enter an integer value for n2: '))

if n1>n2:
    print(f'{n1} is greater than {n2}.')
    print('n1 is greater than n2.')
elif n1<n2:
    print(f'{n1} is lesser than {n2}.')
else:
    print(f'{n1} is equal to {n2}.')

#WAP to check which integer value is the greatest among 4 numbers.

a=int(input('Enter a:'))
b=int(input('Enter b:'))
c=int(input('Enter c:'))
d=int(input('Enter d:'))

if a>b and a>c and a>d:
    print(f'{a} is the greatest number.')
elif b>c and b>d:
    print(f'{b} is the greatest number.')
elif c>d:
    print(f'{c} is the greatest number.')
else:
    print(f'{d} is the greatest number.')

#FIZZBUZZ

#if a number is divisible by 3 print Fizz,
#if the number is divisible by 5 print Buzz,
#if the number is divisible by both 3 & 5 print FizzBuzz.

n=int(input('Enter an integer value: ')) #15
if n%3==0 and n%5==0:
    print('FizzBuzz')
elif n%5==0:

```

```

    print('Buzz')
elif n%3==0:
    print('Fizz')
else:
    print('Not a FizzBuzz number.')

"""#Practise Question on elif statement

1. WAP to find the smallest value among 4 given integer numbers.

2. predict the status of the student result based on the obtained percentage.

3. consider 2 coordinates i.e. x and y and check in which quadrant the point lies.

4. take input of user's age and check if the user is eligible for voting (age>=18) as well as eligible to apply for driving license (age>=21).

5. take input of temperature and predict the weather. if the temperature is below 0 it's 'Freezing' ; 0-15 temperature it's 'Cold' ;
16-30 temperature it's 'Warm' and above 30 temperature it's 'Hot'.

6. Movie Ticket Price
Input age.
Print ticket price:
Below 5 → Free
5-12 → ₹100
13-59 → ₹250
60 and above → ₹150

7. Login System
Assume:

username = "admin"
password = "python123"

Input username and password.
Print:
Login Successful
Incorrect Password
Username Not Found

8. ATM Withdrawal
Assume account balance = ₹20,000.
Input withdrawal amount.
Print:
Invalid amount (≤0)
Insufficient balance
Withdrawal successful
Exact balance withdrawal

9. Triangle Type
Input three sides.
Print:
Equilateral
Isosceles
Scalene
Not a valid triangle

10. Character Type

Input a single character.
Print whether it is:
Uppercase letter
Lowercase letter
Digit
Special character
"""

#consider 2 coordinates i.e. x and y and check in which quadrant the point lies.

#1st quad--both +ve values
#2nd quad--x is -ve and y is +ve
#3rd quad--both -ve values
#4th quad-- x is +ve and y is -ve
#x=0 point lies on x-axis
#y=0 point lies on y-axis
#x=0 and y=0 point lies on th origin

x=int(input('Enter value for x: '))
y=int(input('Enter value for y: '))

if x>0 and y>0:
    print('The point lies in first quadrant.')
elif x<0 and y>0:
    print('The point lies in second quadrant.')
elif x<0 and y<0:
    print('The point lies in third quadrant.')
elif x>0 and y<0:
    print('The point lies in fourth quadrant.')
elif x==0 and y==0:
    print('The point lies at the origin.')
elif x==0:
    print('The point lies in y-axis')
else:
    print('The point lies in x-axis')

#Input three sides. Print: Equilateral Isosceles
#Scalene, Not a valid triangle

a= int(input('Enter side a:'))
b= int(input('Enter side b:'))
c= int(input('Enter side c:'))

if a==b==c:
    print('Equilateral triangle.')
elif a==b or b==c or a==c:
    print('Isosceles Triangle.')
elif a!=b and a!=c and b!=c:
    print('Scalene Triangle')
else:
    print('Not a triangle.')

"""#nested if statement

if-else condition inside another if-else.

"""

#Syntax:
if condition1:
    if condition2:
        #tsb2
    else:

```



```

        #fsb2
    else:
        #fsb1

#WAP to check if a character is Vowel or consonant.

char=input('Enter a character to check:')

if char.isupper() or char.islower():
    if char in 'AEIOUaeiou':
        print('Character is Vowel')
    else:
        print('Character is Consonant.')
else:
    print('The character is not an alphabet.')

#WAP to login to instagram account by entering the proper username and password.

oun='Ishani_12'
opwd='isha@123'

un=input('Enter your username: ')
pwd=input('Enter your password: ')

if oun==un:
    if opwd==pwd:
        print('Log in.')
    else:
        print('Incorrect password.')
else:
    print('Incorrect username.')

#WAP to print the reverse of the string only if it is starting with uppercase character and ending with a digit.

#n='Python_12'
#output: 21_nohtyP'

s=input('Enter a string: ')
if s[0].isupper():
    if s[-1].isdigit():
        print('the reversed string is ',s[::-1])
    else:
        print('The last character is not a digit.')
else:
    print('The starting character is not in uppercase.')

"""#Practice Questions on nested if

1. ATM Withdrawal

A user has:

Account balance = ₹20,000
Correct PIN = 1234

Take PIN and withdrawal amount as input.

Conditions:

First verify the PIN.
If the PIN is correct, check whether the withdrawal amount is less than or equal to the balance.
If yes, deduct the amount and display the remaining balance.
Otherwise, print "Insufficient Balance".

2. Scholarship Eligibility

Take the following inputs:

Percentage
Family income

Conditions:

If percentage is 80 or above, then:
If family income is less than ₹3,00,000, print "Full Scholarship".
Otherwise, print "Half Scholarship".
If percentage is below 80, print "Not Eligible".

3. Movie Ticket Booking

Take:

Age
Ticket availability (yes or no)

Conditions:

If tickets are available:
If age is 18 or above, print "Adult Ticket Booked".
Otherwise, print "Child Ticket Booked".
If tickets are unavailable, print "House Full".

4.Laptop Purchase

Take:

Budget
Is student? (yes or no)

Conditions:

If budget is ₹50,000 or more:
If the user is a student, apply a 10% discount and display the final price.
Otherwise, display the original price.
If budget is below ₹50,000, print "Increase Your Budget".

5. Loan Approval

Take:

Monthly salary
Credit score

Conditions:

If salary is ₹40,000 or above:
If credit score is 750 or above, print "Loan Approved".
Otherwise, print "Loan Rejected due to Low Credit Score".
If salary is below ₹40,000, print "Loan Rejected due to Low Salary".

6. Train Reservation

```

```

Take:

Seat availability (yes/no)
Passenger age
Is senior citizen (yes/no)

Conditions:

If seats are available:
If age is 60 or above:
If senior citizen is yes, give a 40% discount.
Otherwise, give a normal ticket.
Otherwise, book a normal ticket.
Otherwise, print "Waiting List."

#Looping Statements

it is a control statement to control the flow of the execution by repeating the same process again and again for 'n' no of times.

i. while loop: when the times of execution is unknown.

ii. for loop: when the times of execution is known.
"""

#while loop syntax:

initialization [start point]  #compulsary
while condition:
    #tsb
    updation #compulsary

#WAP to print from 1-10.

i=1
while i<=10:
    print('i:',i)
    i+=1  #i=i+1

#WAP to print all the number from 1-20  where the number is multiple of 3.

i=1
while i<=20:
    if i%3==0:
        print('i:',i)
    i+=1

#WAP to print the multiplication table of any given input.

n=int(input('Enter a number: '))
i=1
while i<=12:
    print(f'{n} X {i} = {n*i}')

    i+=1

#WAP to reverse an positive integer number.

#n=123
#rev=321

n=int(input('Enter a value: '))
rev=0  #to store the reversed value.

while n!=0:
    ld=n%10  #123%10=3
    rev=rev*10+ld  #0*10+3=3  #updation
    n=n//10  #123//10=12  #updation
print('Reversed integer is:',rev)

#WAP to find the sum of the individual digits in an integer.

#n=12345
#output=1+2+3+4+5=15

n=int(input('Enter a value: '))
out=0  #to store the sum of the digits

while n!=0:
    ld=n%10
    out=out+ld
    n=n//10

print('The sum of the digits is:',out)

"""#Practice Questions on while loop

1. WAP to find the product of individual digits in a given integer .

2. WAP to find the sum of 'n' natural numbers.

3. WAP to find the sum of the values in the even index position.

Eg: n=123456
1+3+5=9

4. WAP to extract lowercase character from the given string.

5. WAP to find the product of all the float number present in the odd index position in a given tuple .

"""

#WAP to find the product of all the float number present in the odd index position in a given tuple .

t = (10, 2.5, 30, 4.5, 50, 6.2, 70, 8.1)

i = 1
product = 1

while i < len(t):
    if type(t[i]) == float:
        product = product * t[i]
    i = i + 2

print("Product of float numbers at odd index positions =", product)

s = input("Enter a number or string: ")

i = 0
total = 0

while i < len(s):
    if i % 2 == 0:
        total += int(s[i])

```

```

    i += 1

print("Sum of even index digits =", total)

"""#For Loop

Syntax:

for var in collection:
    #statement block

collection can be string,list,tuple,set,dictionary,range

range(): it is used to print number in sequence between the specific limit.

Syntax: range(SV,EV+-1,steps)

SV: starting value
EV: ending value
steps: no of iteration
"""

for i in 'polymorphism':
    print(i,end=' ')

for q in [11,22,33,44,55,66,77,88,99,110]:
    if q%2==0:
        print(q,end=',')

for i in [1,2,3,4,5,6,7,8,9,10]:
    print(i)

for i in range(1,11):
    print(i,end=',')

#print from 20-1

for i in range(20,0,-1):
    print(i,end=',')

for i in range(20,0,-2):
    print(i,end=' ')

#WAP to find the length of a collection without using len().

s=eval(input('Enter a collection:'))
count=0
for i in s:
    count=count+1
print('Length of the collection is:',count)

#WAP to extract the vowels from the given string.

s= input('Enter a string: ') #python
out='' #to store the extracted vowels # 'o'
for i in s:
    if i in 'AEIOUaeiou':
        out=out+i #out=''+o'
print('The vowels are:',out)

#WAP to replace space by underscore in a string input.

#n='python is easy'
#output='python_is_easy'

s= input('Enter a string input:') # 'python is easy'
out='' # 'python_'
for i in s: #i=' '
    if i==' ': # ' '==' '
        out+='_'
    else:
        out=out+i #out+=i #out=''+p'
print(out)

#WAP to check if a given string is palindrome or not without using slicing.

s= input('Enter a string:') #'mom'
rev='' #to store the reversed string
for i in s: #i='m'
    rev=i+rev #rev='m'+om='mom'
print('Reversed string:',rev)

if s==rev:
    print('Palindrome string')
else:
    print('Not a Palindrome String.')

"""#Practice question on for loop

1. WAP to extract all the integer values from a list collection.

list1=[12,2.3,'data',23,4+5j,[1,2,3],100]

2. WAP to extract all the integers which are multiple of 5 and is 3-digit number.

l=[100,25,34,110,20,30,125]
out=[100,110,125]

3. WAP to remove duplicate values from the list.

4. WAP to replace every vowel in a given string with *.

5. WAP to create a new string containing only the characters present at even index positions.

6. WAP to extract all the negative values from a list.

7. WAP to create a new list containing the elements of the original list in reverse order without using slicing or reverse().

8. WAP to create a new string containing only alphabets from an alphanumeric string.

#Split() and Join()
"""

#split(): it break a string into smaller parts into list collection.it is only applied in string collection. it is a builtin function of string.

s='where are you going'.split() #['where','are','you','going']
#out={'where':5,'are':3,'you':3,'going':5}
out={} #to store the result
for i in s:
    out[i]=len(i)
print(out)

n='python is easy to learn'

```

```

out={}
w=n.split()  #['python','is','easy','to','learn']
for i in w:
    out[i]=len(i)
print(out)

#l=['python.py','demo.txt','student.csv','program.css']
#out=['py','txt','csv','css']

l=['python.py','demo.txt','student.csv','program.css','image.jpg']
out=[]
for i in l:  #i='python.py'
    x=i.split('.')  #['python','py']
    out.append(x[-1])
print(out)

#x=['prol.html','file.txt','google.com','yahoo.in']
#out={'html':'prol','txt':'file','com':'google','in':'yahoo'}

x=['prol.html','file.txt','google.com','yahoo.in','outlook.in']
out={}
for i in x:  #i='prol.html'
    r=i.split('.')  #r=['prol','html']
    out[r[1]]=r[0]
print(out)

#join(): it is a built in function of string collection used to join list of string into a single string.

l=['are','you','going','to','have','dinner','now']

#seperator'.join(var_name)

','.join(l)

s='good morning all'
#out='doog gninrom lla'

w=s.split()  #['good','morning','all']
out=[]
for i in w:  #i='good'
    out.append(i[::-1])  #['doog','gninrom','lla']
    res=' '.join(out)
#print(' '.join(out))
print(res)
type(res)

#p='example on for loop'
#out='ee on fr lp'

p='example on for loop'.split()  #['example','on','for','loop']
out=[]
for i in p:
    out.append(i[0]+i[-1])  #i='example'  i[0]='e'+i[-1]='e'='ee'
print(' '.join(out))

#s='abcabacbcc'
#out='a3b3c4'

s='pqrpqrrqqqpppprrrrwww'
out=''

for i in s:
    if i not in out:
        c=s.count(i)
        out+=i+str(c)
print(out)

s='abcaabccbbb'
out={'a':3,'b':5,'c':3}

l=['i@gmail.com','r@yahoo.com','b@qspiders.in']
output=['gmail.com','yahoo.com','qspiders.in']

s='aPPlE#23'
out={'a':'A','P':'p','l':'L','E':'e'}

l=['pl.py','file2.txt','file1.py','google.com','data.txt','yahoo.com']
out={'py':['pl','file1'],'txt':['file2','data'],'com':['google','yahoo']}

s='aPPlE#23'
out='AppLe#23'

"""#Nested for loop

for loop inside another for loop.
"""

#Syntax:

for var1 in collection1:  #outer loop
    for var2 in collection2:  #inner loop
        #statement_block

for i in range(1,6):  #1,2,3,4,5
    for j in range(1,11,3):  #1,4,7,10
        print(i,j)
    print()

l=[12,'program',23.,6+7j,'break',9,'encapsulation']
#out={'program':'oa','break':'ea','encapsulation': 'eauaio'}

out={}
for i in l:  #i='program'
    if type(i)==str:
        vowel=''
        for j in i:  #j='p'
            if j in 'AEIOUaeiou':
                vowel+=j
        out[i]=vowel
print(out)

#pattern programming

'''
* * * * *
'''

for i in range(1,6):
    print('*',end=' ')

'''
* * * * *
* * * * *
'''

```

```

* * * * *
* * * * *
* * * * *
'''

n=int(input('Enter value for rows and columns: '))
for i in range(1,n+1): #outer loop for rows
    for j in range(1,n+1): #inner loop for columns
        print('*',end=' ')
    print() #after completion of the inner loop to take the control in next line

#Primary diagonal

n=int(input('Enter a value: '))

for i in range(1,n+1): #rows
    for j in range(1,n+1):
        if i==j:
            print('*',end=' ')
        else:
            print(' ',end=' ')
    print()

#lower triangle w.r.t. primary diagonal

n=int(input('Enter a value: '))

for i in range(1,n+1): #rows
    for j in range(1,n+1): #columns
        if i>=j:
            print('*',end=' ')
        else:
            print(' ',end=' ')
    print()

#upper triangle w.r.t. primary diagonal

n=int(input('Enter a value: '))

for i in range(1,n+1): #rows
    for j in range(1,n+1): #columns
        if i<=j:
            print('*',end=' ')
        else:
            print(' ',end=' ')
    print()

#secodary diagonal

n=int(input('Enter a value: '))

for i in range(1,n+1): #rows
    for j in range(1,n+1): #columns
        if i+j==n+1:
            print('*',end=' ')
        else:
            print(' ',end=' ')
    print()

#lower triangle w.r.t. secndary diagonal

n=int(input('Enter a value: '))

for i in range(1,n+1): #rows
    for j in range(1,n+1): #columns
        if i+j>=n+1:
            print('*',end=' ')
        else:
            print(' ',end=' ')
    print()

#upper triangle w.r.t. secondary diagonal

n=int(input('Enter a value: '))

for i in range(1,n+1): #rows
    for j in range(1,n+1): #columns
        if i+j<=n+1:
            print('*',end=' ')
        else:
            print(' ',end=' ')
    print()

'''
@
* @
* * @
* * * @
* * * * @
'''

n=int(input('Enter a value:'))
for i in range(1,n+1):
    for j in range(1,n+1):
        if i==j:
            print('@',end=' ')
        elif i>j:
            print('*',end=' ')
        else:
            print(' ',end=' ')
    print()

#print the pattern of hollow square

n=int(input('Enter a value:'))

for i in range(1,n+1):
    for j in range(1,n+1):
        if i==1 or i==n or j==1 or j==n:
            print('*',end=' ')
        else:
            print(' ',end=' ')
    print()

#print the pattern +.

n=int(input('Enter a value:'))

for i in range(1,n+1): #rows
    for j in range(1,n+1): #columns
        if i==n//2+1 or j==n//2+1:
            print('*',end=' ')
        else:

```

```

        print(' ',end=' ')
    print()

#Print the pattern X.

n=int(input('Enter a value:'))

for i in range(1,n+1): #rows
    for j in range(1,n+1): #columns
        if i==j or i+j==n+1:
            print('*',end=' ')
        else:
            print(' ',end=' ')
    print()

```

#Print the pattern T.

```

n=int(input('Enter a value:'))

for i in range(1,n+1): #rows
    for j in range(1,n+1): #columns
        if i==1 or j==n//2+1:
            print('*',end=' ')
        else:
            print(' ',end=' ')
    print()

```

```

'''
1 2 3 4 5
1 2 3 4 5
1 2 3 4 5
1 2 3 4 5
1 2 3 4 5
1 2 3 4 5
'''

```

```

n=5
for i in range(1,n+1): #1,2,3,4,5
    for j in range(1,n+1): #1,2,3,4,5
        print(j,end=' ')
    print()

```

```

n=5
for i in range(1,n+1): #1,2,3,4,5
    for j in range(1,n+1): #1,2,3,4,5
        print(i,end=' ')
    print()

```

```

'''
10
10 11
10 11 12
10 11 12 13
10 11 12 13 14
'''

```

```

n=5 #no.of rows and columns
for i in range(1,n+1):
    k=110
    for j in range(1,n+1):
        if i>=j:
            print(k,end=' ')
            k+=1
        else:
            print(' ',end=' ')
    print()

```

```

'''
I J K L M N O
 I J K L M N
  I J K L M
   I J K L
    I J K
     I J
      I
'''

```

```

n=7
for i in range(1,n+1):
    k=ord('I')
    for j in range(1,n+1):
        if i<=j:
            print(chr(k),end=' ')
            k+=1
        else:
            print(' ',end=' ')
    print()

```

"""#Intermediate Termination in looping statement

1. break: it will stop the execution and will come out of the loop.

2. continue: it is used to skip an iteration

3. pass statement: it is used to make any empty statement block valid.

"""

```

for i in range(1,11):
    print(i,end=' ')
    if i==6:
        break

```

```

for i in range(1,11):
    if i ==5:
        continue
    print('i=',i)

```

```

n=8
if n%2==0:
    pass
else:
    print('ddsff')

```

```

def f_name(a):
    pass
f_name(1)

```

```

class A:
    pass

```

#WAP to ask the user to enter valid username, run the loop continuously until the user gives correct input.

```

oun='Ishani_123'
while True:

```

```

un= input('Enter a username: ')
if oun==un:
    print('Logged In')
    break
else:
    print('Enter correct username.')

#WAP to create the game Guess the Number.

num=67

while True:
    n=int(input('Guess the number: '))
    if n==num:
        print('Congratulation! You have guessed it right.')
        break
    elif n>num:
        print('You have guessed greater number.')
    else:
        print('You have guessed lesser number.')

#How to generate random number.

#Syntax:
#import random
#random.randint(start(a),stop(b))

import random
x=random.randint(1000,9999)
print(x)

#WAP to login to phonepay account by entering the correct OTP.

import random

phno=int(input('Enter your registered number: '))
while True:
    otp=random.randint(1000,9999)
    print('An OTP has been sent to your registered mobile number: ',otp)
    e_otp=int(input('Enter the OTP received in your registered number:'))
    if e_otp==otp:
        print('You have successfully logged in.')
        break
    else:
        print('Incorrect OTP given.')

#to unlock the phone by entering the correct PIN, the program should allow the user to enter wrong pin only 5 times after that user have to wait for 10 sec to
enter correct PIN again.

import time
opin=2026
count=0
while True:
    if count<5:
        pin=int(input('Enter your PIN:'))
        count+=1
        if pin==opin:
            print('Phone Unlocked.')
            break
        else:
            print('Incorrect PIN.')
    else:
        print('Too many wrong inputs. Try again later!!!')

    time.sleep(10)
    count=0

#WAP to extract all the integer values from a given list.

lst=['python',12,78.96,2+3j,23,'continue',78,2.345,45.67,89,'data']
out=[]
for i in lst:
    if type(i)!=int:
        continue
    out.append(i)

print(out)

#WAP to extract all the special characters from a given string.

string=input('Enter a string:')
res=''
for i in string:
    if i.isupper() or i.islower() or i.isdigit():
        continue

    else:
        res+=i
print(res)

"""#Functions

It is a name given to memory block where the instruction is stored and can be used multiple times for performing specific tasks.

There are 2 types of function:

1. built in functions.
2. user defined functions.

built-in functions are those which are predefined by the developers.

len(),type(),bool(),print(),input(),int(),id(),etc....
"""

a=10
b=20
print(a+b)
print('Hello')
c=20
d=30
print(c+d)
x=100
y=200
print(x+y)

def add(a,b):
    print(a+b)

add(10,20)
add(67,78)

```

```
"""#Built in functions
```

```
Categories:
```

```
i. utility
ii. string
iii. list
iv. tuple
v. set
vi. dict
```

i. utility can be used in any form of datatype. Eg: len(), type(), bool(), print(), input(), int(), id(), etc

```
#built in functions of string
"""
```

```
s='GoodMorning@123'
```

```
s.upper() #converts all the characters in uppercase
s.lower() #converts all the characters in lowercase
s.swapcase() #converts all the characters from upper to lower and vice versa
s.replace('@','**') #replace old character with new one
```

```
x='data analysis'
```

```
x.capitalize() #capitalize the 1st character in the string.
x.title()# converts initial character to uppercase.
x.index('a') #it will give the index positin of the 1st char.
x.count('s') #it will give the count of the char in the string.
```

```
"""#built in functions of list"""
```

```
lst=[11,23,34,45,56,78,67,90]
```

```
lst.append(123)
lst.extend([234,456,789,65])
lst.insert(6,900)
lst.pop() #removes the value from the end of the collection
lst.pop(6) #removes the value from the given index position Syntax:var.pop(index_position)
lst.remove(11)
lst.sort() #arranges the values in ascending order
lst.sort(reverse=True)#arrange the values in descnding order
lst.reverse()
lst.index(789) #it will give the index position of the element.
lst.count(123) #it will give the count of the element in the list.
```

```
print(lst)
```

```
"""#built in function of Tuple"""
```

```
t=(10,20,30,40,10,20,30,30,30,50,50,50,50,50)
```

```
t.index(50) #it will give the index position of the element.
t.count(30) #it will give the count of the element in the tuple.
```

```
"""#built in functions on set"""
```

```
s1={12,23,34,45,56,67,78,90}
s2={12,45,78,10,20,67}
#s1.add(77)
#s1.pop()
#s1.remove(90)
```

```
#union(): it combines all the set collection into a new collection
s1.union(s2)
```

```
#intersection(): it will return the common values in a new collection
s1.intersection(s2)
```

```
#difference(): it will fetch the unique values from the 1st collection
s1.difference(s2)
s2.difference(s1)
```

```
#clear(): removes all the values from the collection
s3={11,22,33,44,55,66}
s3.clear()
s3
```

```
"""#built in function of dict"""
```

```
d={'a':'and','b':'break','c':'continue','d':'deploy','e':'end'}
```

```
d.keys()
d.values()
d.items()
d.pop('c')
d.get('e') #it will fetch the value of the given key
d.popitem() #it will remove an item from the end of the collection
d.update({'f':'float','g':'getter','h':'hollow','i':'inheritance'}) #it will update the existing collection with new values
d.clear()
d
```

```
"""#User defined functions
```

it is a function created by the users(programmers) as per their requirement we can use and modify.

```
#Syntax of user defined function:
```

```
def f_name(args):
    #statement_block
    return values
f_name(parameters)
```

def : a keyword to defne the function name.

f_name: function name, a name to identify the function.

args: arguments, variable is called arguments in function

parameters: values in function

return: it returns the values and exit from the function and pass back the value to the caller.

Based on return the categories of user defined functions:

```
i. func without args & without return values.
ii. func without args & with return values.
iii. func with args & without return values.
```



```

iv. func with args & with return values.

#func without args & without return values.
"""

#Syntax:

def f_name():
    #statement_block
    f_name()

#WAP to print the sum of 2 integer values.

def add():
    n1=int(input('Enter n1: '))
    n2=int(input('Enter n2: '))

    total=n1+n2

    print(f'Summation of {n1} and {n2} is {total}.')

add()

#WAP to count the number of occurrence of a given character in a string.

def count_char():
    s= input('Enter a string: ') # 'programming'
    ch= input('Enter the character: ') #'r'
    count=0

    for i in s: #i='r'
        if i==ch:
            count+=1
    print(count)

count_char()

"""#func with args & without return values."""

#Syntax:

def f_name(args):
    #statement_block

    f_name(parameters)

#WAP to add 2 integer numbers

def add_int(x,y): #function declaration

    total=x+y
    print(f'Summation of {x} and {y} is {total}.') #function definition

add_int(12,23) #function call

#WAP to extract strings from the list which are palindrome.

#lst=[12, 'data', 'aba', 67.78, 'star', 56+3j, 'nayan']

def extract_str(lst):
    out=[] #['aba', 'nayan']
    for i in lst: #i='aba'
        if type(i)==str:
            if i==i[::-1]: # 'aba'=='aba'
                out.append(i)
    print(out)
extract_str([12, 'data', 'aba', 67.78, 'star', 56+3j, 'nayan'])

extract_str(eval(input('Enter a collection: ')))

"""# func without args & with return values."""

#Syntax:

def f_name():
    #statement_block
    return value(s)
print(f_name())

#WAP to add 2 integer numbers

def sum_int():
    n1=int(input('Enter n1: '))
    n2=int(input('Enter n2: '))

    total=n1+n2
    return total

print(sum_int())

#WAP to find the sum of integers in a given list.

def sum_list():
    #lst=eval(input('Enter a list collection:'))
    lst=[12, 3.4, 67, 8+9j, 100]
    add=0
    for i in lst:
        if type(i)==int:
            add+=i
    return add
print(sum_list())

"""# func with args & with return values"""

#WAP to add 2 integer values

def add(x,y):
    total=x+y
    return total

print(add(90,67))

#WAP to print the initial index position of a character in a given string.

#n='Encapsulation'
#ch='z'
#output=4

def initial_index(n,ch):
    for i in range(0,len(n)):
        if n[i]==ch:
            return i

```

```

    return 'Character NOT FOUND'

print(initial_index('Encapsulation','i'))

"""#Global and Local Variable

Global variable are the variable which is defined outside the function, these variables are accessed anywhere in the program.

to modify any global variable inside th function we need to have the permission to modify it, to modify we have to use the keyword 'global'.

Local variable are the variable which is declared inside the function, and it can be accessed only inside the function.

to modify local var inside any nested func we have to use 'nonlocal' keyword.

#Difference between gloabl and local variable.

Global var:
.created in main program space.

.scope of the global var is both inside and outside the func.

.to modify global var inside func we have to use 'global' keyword.

Local var:
.created inside the function.

.scope of the var is only inside it's parent func.

.to modify the local var inside nested func we use 'nonlocal' keyword.
"""

#global var:

a=1000
b=2000
def demo():
    global a
    global b
    print(f'The value of a:{a} and b:{b} inside the func.')
    a=1234
    b=2345
    print(f'Modified value of a:{a} and b:{b}.')
demo()
b=7890
print(f'The value of a:{a} and b:{b} outside the func.')

def abcd():
    print(a)
abcd()

#local var:

def abc():
    x=1000
    y=2000
    print(f'The value of x: {x} and y:{y}.')
abc()

#local var in nested func:

def outer():
    m=1000
    n=2000
    print(f'The value of m:{m} & n:{n} inside outer func.')

    def inner():
        nonlocal m
        nonlocal n
        print(f'The value of m:{m} & n:{n} inside inner func.')
        m=9876
        n=5672
        print(f'Modified value of m:{m} & n:{n} inside inner func')
    inner()
outer()

"""#Types of arguments:

1. Positional args: it is the variable which we have to
declare in the function declaration
line. We have to maintain the order.

2. Keyword argument: it is present in the function call. we donot have
to maintain the order.

3. Default argument: by default we are going to assign a value in the
argument which is going to be used
multiple time. it will be assigned in function declaration line.

4. Variable length argument
"""

#positional args:

def info(name,city):
    print(f'Name:{name}, City:{city}')
info('Ishani','Kolkata')

#keyword args:

def details(name,subject):
    print(f'Name of the student is {name}, opted for the subject {subject}.')

details(subject='Python',name='Amrutha')

#default args:

def employee(name,job='SE'):
    print(name,job)
employee('Varshini')
employee('Sumbul','ASE')
employee('Amrutha')

#Variable length args:

def f_name(*args,**kwargs):
    #statement_block
    f_name(val1,val2,...,valn,k1=v2,k2=v2,...kn=vn)

def fact(n):
    fact=1 #120

    for i in range(1,n+1): #1,2,3,4,5

```

```

        fact=fact*i

    return fact

print(fact(5))

"""#Practice Questions on user defined func

1. Write a function that takes two numbers as arguments and prints the larger number.

2. Write a function that takes a string and returns the first character that appears only once.

3. Write a function that takes a list and moves all 0s to the end while maintaining the order of the other elements.

4. Write a function that takes a sentence and returns the longest word.

5. Write a function that takes a list and returns a new list after removing duplicates.[donot use set]
"""

```

#4

```

def longest_word(sentence):
    words = sentence.split()

    longest = words[0]

    for word in words:
        if len(word) > len(longest):
            longest = word

    return longest

print(longest_word("Python makes programming interesting"))

```

"""#Packing and Unpacking

It is a process of wrapping up of multiple values in a single variable.

There are 2 types:

1. Single/Tuple packing
2. Double/ Dictionary packing

1. Single/tuple packing:where we can group individual data item in the form of tuple using the packing concept.

By default all the values will be packed in a single var in the tuple format, as it is the most secured datatype.

2. Double/dictionary packing: It is used to wrap key-value pair.

"""

#without packing

```

def demo(a):
    print(a)

demo(1,2,3,4,5,6,7,8)

```

#with packing

#Syntax:

```

def f_name(*args):
    #statement_block

f_name(v1,v2,....,vn)

#single/tuple packing
def pack(*x):
    print(x)
    print(type(x))
pack(10,22,34,56,90,100,78,56)

```

#double/dictionary packing

#Syntax:

```

def f_name(**args):
    #statement_block
f_name(k1=v1,k2=v2,....,kn=vn)

def double_pack(**k):
    print(k)
    print(type(k))

double_pack(a='abstract',b='break',c='continue',d='default')

```

#merging of single and double packing

#Syntax:

```

def f_name(*args,**kwargs):
    #statement_block

f_name(v1,v2,....,vn,k1=v1,....kn=vn)

def pack(*x,**y):
    print('x: ',x)
    print('y: ',y)

pack(10,20,30,40,50,a='and',b='break',c='continue')

```

"""#Unpacking

It is a process of dividing the collection into n no of individual values.

We can unpack any form of datatype.

"""

#Syntax of unpacking:

```

def f_name(args):
    #statement_block
f_name(*collection)

def unpack(a,b,c,d,e,f):
    print('a: ',a)
    print('b: ',b)
    print('c: ',c)
    print('d: ',d)
    print('e: ',e)
    print('f: ',f)

```

```
#unpack(*'python')
#unpack(*[10,20,30,40,50,60])
unpack(*range(100,106))

"""#Recursion

When a function calls itself inside it then we call it as recursive
call/func; while the whole process is called as Recursion.
"""
```

#Syntax:

```
def f_name(args): #func declaration
    #statement_block
    f_name() #recursive call
```

f_name(parameters) #func calling

```
def demo():
    print('Hello World.')
    demo() #recursive call
demo()
```

#to make recursion work properly

#syntax:

```
def f_name(args):
    base condition
    return value(s)
```

```
    #statement_block
    recursive call
```

f_name(parameters)

```
def recursion(n):
    if n==0:
        return
    print('n: ',n)
```

```
    recursion(n-1)
recursion(20)
```

#Factorial of a number

```
def fact(n):
    if n==1:
        return 1
    return n*fact(n-1) #5*fact(5-1)=5*fact(4)
n=int(input('Enter value to find factorial: '))
print(fact(n))
```

#Power of a number

#2**5=32

```
def pow(base,expo): #func declaration
    if expo==0:
        return 1
    else:
        return base*pow(base,expo-1) #recursive call
base=int(input('Enter base value:'))
expo=int(input('Enter exponent: '))

print(pow(base,expo)) #func call
```

#Fibonacci series

#0 1 1 2 3 5 8 13

```
def fibo(n):
    if n==0:
        return 0
    elif n==1:
        return 1
    else:
        return fibo(n-1)+fibo(n-2)
```

```
terms=int(input('Enter no of terms: '))
print('Fibonacci Series:') #optional
for i in range(terms):
    print(fibo(i),end=' ')
```

"""#Introduction to OOP

OOP- Object Oriented Programming

Loop: code repetition

Function: code repetition reuseability

OOP: code repetition
reuseability
proper sturcture
code flexibility
collaborate
real time entity(object)

OOP is used to work with real time entity.

class: It is called a blueprint.

It is basically a container used to store data and can be utilized multiple times.

Object: it is an exact copy of class.

method: it is a function inside the class, which is dependent on the class.

#Types of States/Properties:

There are 2 type:

1. class/static/generic properties: these are the properties which is common for all the objects
2. object/ specific properties: these are the properties which is different for all the objects.

__init__: also called as constructor method/initialization method.
It is Called by default once an object is created.
According to Industrial Standard Rule the first term should be 'self'.

self: it is a parameter which is used to store the address of the object.

a class can have multiple methods and objects; but only one constructor method.
"""

#Syntax of class and object

```
class Class_name:
    #class properties/states
```

```
    #method
    def method_name(self,object properties):
        #statement_block
```

```
#object creation
object_name=Class_name(args)
```

```
class Demo:
    pass
```

```
ob1=Demo()
ob2=Demo()
```

```
print(ob2)
```

```
class School:
    #properties
    s_name='ABC Institute'
    s_loc='Kolkata'
    s_website='www.abcinstitute.com'
    s_time='7am-3pm'
```

```
#method creation with object properties
def __init__(self,name,roll,standard,sec,phno,marks):
    self.name=name
    self.roll=roll
    self.standard=standard
    self.sec=sec
    self.phno=phno
    self.marks=marks
```

```
#object creation [student]
s1=School('Atul',101,12,'A',9876543210,80)
s2=School('Amrutha',102,12,'A',8765432190,85)
s3=School('Varshini',103,12,'A',8765432190,75)
s4=School('Selva',104,12,'A',8765432190,90)
```

```
print(s1.name,s1.marks)
print(s2.name,s2.phno)
print(s3.name,s3.standard)
print(s4.name,s4.roll,s4.s_name)
```

```
print(School.s_name,School.s_loc)
```

#Create a class called Bank and object will be cusomer. Take atleast 4 class properties and atleast 5 object properties. print the object properties as per your own wish.

"""#Types of Methods

There are 3 types of methods:

1. object method: these are used to modify or access the object properties. it is compulsory to store the first parameter as the 'self'.

2. class method: these are used to access or modify the class properties. to access or modify the class properties we have to use 'cls' as the first parameter, @classmethod [decorator].

3. static method: it is neither related to class members or object members but will act as a supportive method. to create static method we have to use @staticmethod [decorator].
"""

```
class Company:
    #class properties
    c_name='TCS'
    c_loc='Bangalore'
    login_hour='9hours'

    #constructor method with object properties

    def __init__(self,name,eid,contact,email,job,salary):
        self.name=name
        self.eid=eid
        self.contact=contact
        self.email=email
        self.job=job
        self.salary=salary
#access of object properties
    def display_details(self):
        print(self.name,self.eid,self.salary,self.job,self.email,self.contact)
#modify object properties
    def ch_salary(self,new):
        self.salary=new
        print(self.salary)
```

```
#class method to access the class property
    @classmethod
    def display(cls):
        print(cls.c_name,cls.c_loc,cls.login_hour)
```

```
#class method to modify the class property
    @classmethod
    def ch_location(cls,new):
        cls.c_loc=new
        print(cls.c_loc)
```

```
#static method
    @staticmethod
    def annual_salary_calc(monthly_salary):
        annual_salary=monthly_salary*12
        return annual_salary
```

```
#object creation
```

```
E1=Company('Atul','TCS1001',9876543210,'atul@tcs.in','Developer',80000)
E2=Company('Varshini','TCS1123',9876543211,'varshini@tcs.in','Tester',85000)
E3=Company('Brindha','TCS1234',9876543210,'brindha@tcs.in','Analyst',90000)
```

```
#Object Method
#E1.display_details()
#E2.display_details()
#E3.display_details()
```

```
#E2.ch_salary(95000)
#E2.display_details()
```

```

#Class Method

E1.display()
E3.display()
E3.ch_location('Kolkata')

#Static Method

E3.annual_salary_calc(90000)

E3.display_details()

"""#Practice Questions on methods

Q1
Create a class Student:

object method → display details
Class method → change school name
Static method → check if marks > 40 (pass/fail)

Q2
Create a class Bank:

Object method → deposit/withdraw
Class method → change interest rate
Static method → calculate simple interest

Q3
Create a class Product:

Object method → calculate total price
Class method → apply discount to all products
Static method → check if price is valid (>0)

Q4
Create a class Circle:

Object method → calculate area
Class method → change default radius
Static method → return value of π (3.14)
"""

class Circle:

    # Class property
    r= 5

    # Object method
    def __init__(self, radius):
        self.radius = radius

    def calculate_area(self):
        area = 3.14 * self.radius * self.radius
        return area

    # Class method
    @classmethod
    def change_default_radius(cls, new_radius):
        cls.r = new_radius

    # Static method
    @staticmethod
    def get_pi():
        return 3.14

# Creating objects
c1 = Circle(10)
c2 = Circle(7)

# Calling object method
print("Area of Circle 1:", c1.calculate_area())
print("Area of Circle 2:", c2.calculate_area())

# Calling class method
Circle.change_default_radius(10)
print("New Default Radius:", Circle.r)

# Calling static method
print("Value of Pi:", Circle.get_pi())

"""There are 4 pillar of OOP in python:

1. Inheritance
2. Encapsulation
3. Abstraction
4. Polymorphism

#Inheritance

it is a process where a child class derives the properties from parent class is called as inheritance.

child class is also known as derived class/ sub class

parent class is also known as super class/ base class.

There are 5types:

1. Single level inheritance
2. Multi level inheritance
3. Multiple inheritance
4. Hierarchical inheritance
5. Hybrid inheritance

object creation is always be done for the child class, as parent class cannot acces the properties of the child class.
"""

#Single level inheritance: one child class who will derive from one parent class.

#Syntax:
'''
class Parent_class:
    #SB

class Child_class(Parent_class):

```

```

#SB
'''

#Code example:

class Student: #parent class
    def __init__(self,name,addr,s,dob):
        self.name=name
        self.addr=addr
        self.dob=dob

    def display(self): #object method to access
        print(f'Name: {self.name}, Address: {self.addr}, Date of Birth: {self.dob}')

class School(Student): #child class
    def __init__(self,name,addr,s,dob,s_name,marks):
        super().__init__(name,addr,s,dob) #constructor chaining
        self.s_name=s_name
        self.marks=marks
    def display_details(self):
        super().display() #method chaining
        print(f'School name: {self.s_name}, Percentage: {self.marks}')

#object creation

st1=School('Sneha','Kolkata','25-03-2000','SXI','88%')

st1.display()
print()
st1.display_details()

#Multi level Inheritance: a child class will derive the property from it's parent class which in turn will beocme the parent class of the next derived class.
[Grandfather--> Father--> Child]

#Syntax:
'''
class Parent_class1:
    #SB
class Parent_class2(Parent_class1):
    #SB

class Child_class(Parent_class2):
    #SB
'''

#Code Example:

class Student: #parent class
    def __init__(self,name,addr,s,dob):
        self.name=name
        self.addr=addr
        self.dob=dob

    def display(self): #object method to access
        print(f'Name: {self.name}, Address: {self.addr}, Date of Birth: {self.dob}')

class School(Student): #child class
    def __init__(self,name,addr,s,dob,s_name,marks):
        super().__init__(name,addr,s,dob) #constructor chaining
        self.s_name=s_name
        self.marks=marks
    def display_details(self):
        super().display() #method chaining
        print(f'School name: {self.s_name}, Percentage: {self.marks}')

class College(School):
    def __init__(self,name,addr,s,dob,s_name,marks,clg_name,cgpa):
        super().__init__(name,addr,s,dob,s_name,marks)
        self.clg_name=clg_name
        self.cgpa=cgpa
    def disp(self):
        super().display_details()
        print(f'College Name: {self.clg_name}, CGPA:{self.cgpa}')

#object creation
st1=College('Sneha','Kolkata','25-03-2000','SXI','88%','KIIT',8.39)
st1.display()
print()
st1.display_details()
print()
st1.disp()

#Multiple Inheritance: one child class will inherite from multiple parent class.

#Syntax:
'''
class Parent_class1:
    #SB
class Parent_class2:
    #SB
.
.
.
class Parent_class_n:
    #SB

class Child_class(Parent_class1,Parent_class2,...Parent_class_n):
    #SB
'''

#Code example:

class School:
    def __init__(self,s_name,percentage):
        self.s_name=s_name
        self.percentage=percentage
    def display_school(self):
        print(f'School Name:{self.s_name}, Percentage Obtained:{self.percentage}')

class College:
    def __init__(self,c_name,cgpa):
        self.c_name=c_name
        self.cgpa=cgpa
    def display_college(self):
        print(f'College Name:{self.c_name}, CGPA Obtained:{self.cgpa}')

class University:
    def __init__(self,uni_name,CGPA):
        self.uni_name=uni_name
        self.CGPA=CGPA
    def display_university(self):

```

```

    print(f'University Name:{self.uni_name}, CGPA Obtained:{self.CGPA}')

class Student(School,College,University): #child class
    def __init__(self,name,addr,do,contact,s_name,percentage,c_name,cgpa,uni_name,CGPA):

        #calling parent class constructor
        School.__init__(self,s_name,percentage)
        College.__init__(self,c_name,cgpa)
        University.__init__(self,uni_name,CGPA)

        self.name=name
        self.addr=addr
        self.do=do
        self.contact=contact
    def display(self): #object method to access the property of Student class
        print(f'Student Name: {self.name}, Address:{self.addr}, Date of Birth: {self.do}, Mobile Number: {self.contact}')
    def display_all(self): #object method to access all the details of the student
        super().display_school()
        super().display_college()
        super().display_university()
        print(f'Student Name: {self.name}, Address:{self.addr}, Date of Birth: {self.do}, Mobile Number: {self.contact}')
#object creation

n=Student('Rohan','Kolkata','04-04-2000',9876543210,'ABC School','92%','Techno University',9.2,'KIIT University',9.01)
'''
n.display_school()
print()
n.display_college()
print()
n.display_university()
print()
n.display()
'''

n.display_all()

#Hierarchical Inheritance: on parent class and multiple child class.

#Syntax

'''
class Parent_class:
    #SB

class Child_class_1(Parent_class):
    #SB
.
.
class Child_class_n(Parent_class):
    #SB
'''

#code example:

class BankAccount:
    def __init__(self,name,addr,balance):
        self.name=name
        self.addr=addr
        self.balance=balance #5000

    def display(self): #object method display customer details
        print(f'Account Holder name:{self.name}, Address:{self.addr}, Balance:{self.balance}')

    def display_balance(self):
        print(f'Rs.{self.balance}')

    def deposit(self,amount):
        self.balance+=amount #5000+2000=7000
        print(f'Rs.{amount} is deposited successfully.')

    def withdraw(self,amount):
        if amount<=self.balance:
            self.balance-=amount
            print(f'Rs.{amount} debited from your account.')
        else:
            print('Insufficient balance')

class SavingsAccount(BankAccount):
    def add_interest(self):
        interest=self.balance*0.04 #7000*0.04=280.0
        self.balance+=interest
        print(f'Rs.{interest} interest credited in your account.')

class CurrentAccount(BankAccount):
    def __init__(self,name,addr,balance,limit):
        super().__init__(name,addr,balance)
        self.limit=limit
    def overdraft(self,amount):
        if amount<=self.limit:
            self.balance-=amount
            print(f'Rs.{amount} overdrafted')
        else:
            print('Limit Exceeded.')

#object creation
'''
s=SavingsAccount('Ishani','Kolkata',100000)
s.display()
print()
s.deposit(55000)
print()
s.display_balance()
print()
s.withdraw(1600)
print()
s.display_balance()
print()
s.add_interest()
print()
s.display_balance()
'''

c=CurrentAccount('Sneha','Delhi',500000,35000)
c.display()
c.deposit(22300)
c.withdraw(90000)
c.overdraft(3600)
c.display_balance()

#Hybrid Inheritance: combination of all the inheritance.

#code example

```



```

class BankAccount: #parent class
    def __init__(self,name,balance):
        self.name=name
        self.balance=balance

    def display(self):
        print(f'Account Holdername:{self.name}, Balance:{self.balance}')
    def display_balance(self): #object method to display balance in acc
        print(f'Rs.{self.balance}')

    def deposit(self,amount):
        self.balance+=amount #5000+9000=14000
        print(f'Rs.{amount} is deposited successfully.')

    def withdraw(self,amount):
        if amount<=self.balance:
            self.balance-=amount
            print(f'Rs.{amount} debited from the account')
        else:
            print('Insufficient Balance')

class SavingsAccount(BankAccount):
    def add_interest(self):
        interest=self.balance*0.04 #14000*0.04=560
        self.balance+=interest
        print(f'Rs.{interest} interest added in your account')

class CurrentAccount(BankAccount):
    def __init__(self,name,balance,limit):
        super().__init__(name,balance)
        self.limit=limit
    def overdraft(self,amount):
        if amount<=self.limit: #5000<=20000
            self.balance-=amount
            print(f'Rs.{amount} overdraft used.')
        else:
            print('Overdraft exceeded')

class PremiumAccount(SavingsAccount, CurrentAccount):
    def __init__(self, name, balance, overdraft_limit):
        SavingsAccount.__init__(self, name, balance)
        CurrentAccount.__init__(self, name, balance, overdraft_limit)
    def premium_benefits(self):
        print("The account is eligible for premium benefits.")

```

#Object creation

```

#savings account
'''
s1=SavingsAccount('Sneha',90000)
s1.display()
s1.deposit(20000)
s1.display_balance()
s1.withdraw(7000)
s1.display_balance()
s1.add_interest()
s1.display_balance()
'''

#current account
'''
c1=CurrentAccount('Sumonta',600000,35000)
c1.deposit(15000)
c1.display()
c1.withdraw(50000)
c1.overdraft(3100)
c1.display_balance()
'''

```

```

#premiun account
p=PremiumAccount('Rohan',700000,40000)
p.premium_benefits()
p.add_interest()
p.overdraft(4500)
p.deposit(100000)
p.withdraw(90000)
p.display_balance()

```

""""#Polymorphism

poly-- many
morphism-- forms

It is a process where a method/operator work on two / more different operations is called Polymorphism.

****overloading is overflowing.**
****overriding is replacing previous value with new values.**
 There are 2 types:

1. Method overloading (overridden): It is aprocess where the previous address of the method is replaced with new one.
2. Operator overloading: It is a process of making an operator work on object of user-defined datatype.

Monkey Patching: It is a process where a third variable will act as a method/function name.

"""

#Method overloading:

```

def demo():
    print('Ploymorphism')
abc=demo
def demo(a,b):
    print(a+b)
mid=demo
def demo(a,b,c):
    print(a*b*c)

```

```

demo(2,3,4)
mid(56,90)
abc()

```

#code example

#Employee Salary

```

class Employee:
    def check_salary(self):
        pass

class Manager(Employee):
    def check_salary(self):
        return 'Manager draws a salary if Rs.90000'

```

```

class Developer(Employee):
    def check_salary(self):
        return 'Developer draws a salary if Rs.80000'

class Intern(Employee):
    def check_salary(self):
        return 'Intern draws a salary if Rs.30000'

emp=[Manager(),Developer(),Intern()]
for i in emp:
    print(i.check_salary())

class A:
    def __init__(self,a):
        self.a=a

    def __add__(self,other):
        return self.a+other.a

    def __sub__(self,n):
        return self.a-n.a

ob1=A(100)
ob2=A(200)
print(ob1+ob2)    #user-defined datatype.
print(ob2-ob1)

```

#code example on operator overloading:

#Shopping Cart

```

class ShoppingCart:
    def __init__(self,items,price):
        self.items=items
        self.price=price

        self.price=self.price-(self.price*0.05)

    def __add__(self,other):
        total_items=self.items+other.items #calculate no of items
        total_bill=self.price+other.price #calculate total amt of bill
        return ShoppingCart(total_items,total_bill)

```

#object creation

```

biscuits=ShoppingCart(3,150)
juice=ShoppingCart(4,360)
chips=ShoppingCart(5,100)
soap=ShoppingCart(4,200)

total=biscuits+juice+chips+soap

print('No of product:',total.items)
print('Total Amount:Rs.',total.price)

```

""""#Abstraction

It is a process where the essential features of an object is shown while the internal features are kept hidden.

ATM Machine

There are 3 terms:

1. abstract method: it will have method declaration but no defination.
2. abstract class: it is a class which should have atleast one abstract method in it.

Note: abstract class cannot have an object

3. concrete class: it is a class which is not having any abstract method in it.
- """

#Syntax of abstract class and method

```

class Class_name:
    def m_name(self,args): #func declaration
        pass

```

#Syntax concrete class:

```

class Class_name:
    def m_name(self,args):
        #statement_block

```

#object creation

```

object_name=Class_name(args)

```

#How to implement Abstraction?

```

#abc: abstract base class (module)
#ABC: Abstract Base Class (special class)
#@abstractmethod: A decorator indicating abstract methods.

```

```

from abc import ABC,abstractmethod

```

```

class Class_name(ABC):
    @abstractmethod
    def m_name(self,args):
        pass

```

#code example:

```

from abc import ABC,abstractmethod
class Vehicle(ABC):
    @abstractmethod
    def start(self):
        pass
    @abstractmethod
    def stop(self):
        pass

```

#concrete class

```

class Bike(Vehicle):
    def start(self):
        print('Bike is moving.')

    def stop(self):
        print('Bike has stopped.')

```

#concrete class

```

class Car(Vehicle):
    def start(self):
        print('Car is moving.')

    def stop(self):
        print('Car has stopped.')

#object creation

b=Bike()
c=Car()

b.start()
b.stop()

c.start()
c.stop()

#An e-commerce app should accept different payment methods(Credit Card,UPI Payment,NetBanking), but the app should not care the payment method, create an abstract class.

from abc import ABC,abstractmethod

class Payment(ABC):
    @abstractmethod
    def pay(self,amount):
        pass

class CreditCard(Payment):
    def pay(self,amount):
        print(f'Paid Rs.{amount} using Credit Card.')

class UPIPayment(Payment):
    def pay(self,amount):
        print(f'Paid Rs.{amount} using UPI Payment.')

class NetBanking(Payment):
    def pay(self,amount):
        print(f'Paid Rs.{amount} using NetBanking.')

def checkout(payment,amount):
    payment.pay(amount)

#object creation

credit=CreditCard()
checkout(credit,10000)

upi=UPIPayment()
checkout(upi,10000)

"""#Encapsulation

It is a process of protecting the data(object memeber) inside the class.

Encapsulation is also known as access specifier.

Based on the term access specifier it can be categoried into following categories:

1. public access specifier: it is a type where we can modify or access the object memebers anywhere in the class.

Syntax: var=value

2. protected access specifier: it is a type where the object memebrs can be modified/ accessed only inside the class/subcalss.

Syntax: _var=value (single underscore)

3. private access specifier: it is a type where the object memebers can be accessed/modified only inside the respective class.

Syntax: __var=value (double underscore)
"""

#public access specifier:

class Student:
    def __init__(self,name,marks):
        self.name=name
        self.marks=marks #public access specifier

    def display(self):
        print(f'Student Name:{self.name} , Percentage: {self.marks}.')

    def ch_marks(self,new_marks):
        self.marks=new_marks

#object creation

s1= Student('Ishani',91)
s1.display()
s1.ch_marks(95)
s1.display()

#protected access specifier:

class Student:
    def __init__(self,name,marks):
        self.name=name
        self._marks=marks #protected access specifier

    def display(self):
        print(f'Student Name:{self.name} , Percentage: {self._marks}.')

class College(Student):
    def show(self):
        print(f'Student Name:{self.name} , Percentage: {self._marks}.')

#object creation

s2=College('Rohan',82)

s2.display()

print(s2._marks) #access outside the class...recommended not to access outside to avoid data loss.

s2.show()

#private access specifier:

class Student:
    def __init__(self,name,marks):
        self.name=name

```

```

        self.__marks=marks    #private access specifier

def __display(self):
    print(f'Student Name:{self.name} , Percentage: {self.__marks}.')

def show(self):
    self.__display()

#object creation

s3=Student('Sneha',78)
#print(s3.__marks)    #error since it is a private specifier

#s3.__display()    #private specifier not accessble

s3.show()

#to modify private access specifier we have 2 ways:

#get and set method (getter & setter method)

#Syntax:

class Class_name:
    def __init__(self,args):
        self.__var=value

    def set_var(self,new): #modify
        return self.__var=new

    def get_var(self): #access
        return self.__var

#code:

class Student:
    def __init__(self,marks):
        self.__marks=marks    #private specifier

    def set_marks(self,new_marks): #modify
        if new_marks>0:
            self.__marks=new_marks
    def get_marks(self): #access
        return self.__marks

#object creation
s4=Student(85)
print(s4.get_marks())

s4.set_marks(95)
print(s4.get_marks())

#property decorator
'''
1. it is compulsory to use getter and setter method.

2. @property [decorator] should be used with the method which will return the value. [accessing the object member]

3. both the method name should be same with the var name.

4. for method which we will used to modify values we have to use @var.setter [decorator].
'''

#code

class Student:
    def __init__(self,marks):
        self.__marks=marks

    @property
    def marks(self): #access
        return self.__marks

    @marks.setter
    def marks(self,new_marks): #modify
        if new_marks>0:
            self.__marks=new_marks
        else:
            print('invalid')

#object creation

s5=Student(78)
#print(s5.__marks)

print(s5.marks)

s5.marks +=5
print(s5.marks)

class Company:
    def __init__(self,salary,contact):
        self.__salary=salary
        self.__contact=contact

    def set_salary(self,new_sal): #modify
        self.__salary=new_sal

    def get_salary(self): #access
        return self.__salary

    def set_contact(self,new_no): #modify
        self.__contact=new_no

    def get_contact(self): #access
        return self.__contact

#object creation

emp=Company(20000,9876543210)
print(emp.get_salary())
emp.set_salary(35000)
print(emp.get_salary())

print(emp.get_contact())
emp.set_contact(1234567890)
print(emp.get_contact())

"""#ADVANCE CONCEPTS OF PYTHON

#Functional Programming

```

```

to make less lines of instructions and to increase the efficiency of the program.
"""

#lambda: it is a keyword used to perform small/basic programs in Python.
#It is an anonymous function.

#Syntax: var_name=lambda args: expression [simple program like simple if.]

#WAP to find if a no is even.

def even(n):
    if n%2==0:
        return 'Even'
    else:
        return 'Odd'

print(even(8))

#var_name=lambda args: expressions      #using this syntax output will be in True or False

res=lambda n: n%2==0
print(res(9))

#WAP to check if a given string starts with a vowel.

out=lambda n: n[0] in 'AEIOUaeiou'
print(out('Abstraction'))

# var_name=lambda args: TSB if condition else FSB [if-else]

#WAP to return n+10 if a no is multiple of 5 else return n-10.

out=lambda n: n+10 if n%5==0 else n-10
print(out(101))

#WAP to check if a given string is palindrome, then return the string as it is else return the reversed string.

out=lambda s: s if s==s[::-1] else s[::-1]
print(out('parrot'))

#MAP Function: for loop statements. it is anonymous function it doesnot have a structure of its own.

#Syntax: var_name=map(f_name,collection)

#print(list/tuple/dict(var_name))

#WAP to find the square of all the nos from 1-10.

square=map(lambda n:n**2,range(1,11))
print(list(square))

#WAP to find the factorial of all the integers in a given collection.

import math

fact=map(math.factorial,[2,3,4,5,6])
print(list(fact))

#output={1:1,2:2,3:6,4:24,5:120}

out=map(lambda n:(n,n**3),range(1,6))
print(dict(out))

#filter function: for loop statements. it is anonymous and doesnot have a structure.

#Syntax: var_name=filter(f_name,collection)

#WAP to extract the string data from a collection only if it starts with lowercase char and ends with uppercase char.

lst=[12,'Data',2+3j,'abstracT',67.89,'publiC','private','Break','continue']

result=filter(lambda n:type(n)==str and n[0].islower() and n[-1].isupper(),lst)

print(list(result))

#WAP to extract the integer values which are multiple of 3 from 1-20.

x=filter(lambda n:n%3==0,range(1,21))
print(list(x))

"""#Practice Questions on lambda,map,filter:

1. Create a lambda function that accepts a string and returns its last character.
2. Create a lambda function that accepts an employee's salary and calculates a 15% bonus.
3. Create a lambda function that checks whether a given string is a palindrome.
4. Create a lambda function that accepts length and width and returns the area of the rectangle.
5. Create a lambda function that accepts an email address and returns only the domain.
6. Given a list of words, use map() and lambda to create a list containing the length of each word.
7. Given a list of distances in kilometers, use map() and lambda to convert every value into miles.
8. Given a list containing full names, use map() and lambda to extract only the first name.
9. Given a list of birth years, use map() and lambda to calculate the age of each person.
10. A student has marks out of 500. Given a list of total marks obtained, use map() and lambda to calculate the percentage.
11. Given a list of words, use filter() and lambda to extract words whose length is greater than 5.
12. Use filter() and lambda to extract numbers that are divisible by both 3 and 5.
13. Given a list of email addresses, use filter() and lambda to extract only Gmail addresses.
14. Given employee performance scores, extract employees whose score is 80 or above.
15. Given a list of passwords, use filter() and lambda to select passwords having at least 8 characters.

#Comprehension

a new collection will be created using less lines of codes.

It is applicable in mutable datatype.

3 types:

1. list comprehension
2. set comprehension

```

```

3. dict comprehension
"""

#List comprehension:

#Syntax:

#1. var_name=[var1 for var in collection] #var1 is to store the value #basic syntax
#2. var_name=[var1 for var in collection if condition] #simple if
#3. var_name=[TSB if condition else FSB for var in collection] #if-else
#4. var_name=[(var1,var2) for var1 in collection1 for var2 in collection2] #nested statement
#var_name=[var1 for var in collection]

#WAP to create a list collection of 1-20.

out=[i for i in range (1,21)]
print(out)

#var_name=[var1 for var in collection if condition]

#WAP to create a list of square nos which is multiple of 3 in the range of 1-20.

res=[p**2 for p in range(1,21) if p%3==0]
print(res)

#var_name=[TSB if condition else FSB for var in collection]

#WAP to create a list collection which will store the square of even nos and cube of odd nos.

x=[i**2 if i%2==0 else i**3 for i in range(1,11)]
print(x)

#var_name=[(var1,var2) for var1 in collection1 for var2 in collection2]

students=['Sneha','Rashmi','Puja']
subjects=['Science','English','History']

#out=[('Sneha','Science'),('Sneha','English'),('Sneha','History'),...]

x=[(p,q) for p in students for q in subjects]
print(x)

#Set comprehension: unordered output, no duplicate values

#Syntax:

#1. var_name={var1 for var in collection} #var1 is to store the value #basic syntax
#2. var_name={var1 for var in collection if condition} #simple if
#3. var_name={TSB if condition else FSB for var in collection} #if-else
#4. var_name={(var1,var2) for var1 in collection1 for var2 in collection2} #nested statement
#WAP to find the square root of all the integers from 1-5.

import math
sq_root=[math.sqrt(i) for i in range(1,6)]
print(sq_root)

#Dictionary Comphrension:

#1. var_name={k:v for k,v in collection} #basic syntax
#2. var_name={k:v for k,v in collection if condition} #simple if
#3. var_name={k:v for k,v in zip(collection1,collection2)} #nested collection
#zip(): it will iterate through mulitple collection at the same time.[ nested collection] zip(collection1,collection2)

keys=['name','city','job','salary']
values=['Rashmi','Kolkata','Analyst','70000']

#output={'name':'Rashmi','city':'Kolkata','job':'Analyst','salary':70000}

res=zip(keys,values)
print(dict(res))

#var_name={k:v for k,v in collection if condition}

s='what are you doing'.split()
#out={'are':3,'you':3,'doing':5}

out={k:len(k) for k in s if len(k)%2!=0}
print(out)

#var_name={k:v for k,v in collection}

#output={1:1,2:8,3:27,4:64,5:125}

res={k:k**3 for k in range(1,6)}
print(res)

#var_name={k:v for k,v in zip(collection1,collection2)}

city=['Jaipur','Kolkata','Delhi']
name=['Pink City','City of Joy','Heart of India']

#out={'Jaipur':'Pink City','Kolkata':'City of Joy','Delhi':'Heart of India'}

out={k:v for k,v in zip(city,name)}
print(out)

"""#Practise Questions on Comphrension

#List Comprehension

1. Given a list of integers, create a new list containing the square of
only odd numbers, but skip numbers divisible by 5.
2. From two lists a = [1,2,3] and b = [4,5], create a list of tuples (x, y)
such that their sum is even.
3. Given a sentence, create a list of words that start with a vowel and
are longer than 3 characters.
4. Given a list of numbers, replace even numbers → 'even', odd
numbers → 'odd', but if divisible by 7 → 'lucky'.
5. Flatten a nested list [[1,2,3],[4,5],[6,7,8]] and keep only numbers
greater than 4.

```

#Set Comprehension

```
1. From a list with duplicates, create a set of squares of numbers
divisible by 3.
2. Given a string, create a set of unique consonants only (ignore
vowels and spaces).
3. From numbers 1 to 50, create a set of prime numbers.
4. From nested list [[2,4],[4,6],[6,8]], create a set of all even numbers
greater than 4.
5. From a list of words, create a set of lengths of words that contain
the letter 'a'.
```

#Dictionary Comprehension

```
1. Create a dictionary where key = number and value = square, only
for numbers divisible by 4 from 1 to 20.
2. Given a sentence, create a dictionary where key = word and value =
length, excluding words shorter than 4 characters.
3. Given a list of numbers, create a dictionary where key = number
and value = 'even' or 'odd', skipping numbers divisible by 5.
4. Given a string, create a dictionary of character frequencies (ignore
spaces).
5. Given keys = ['a','b','c','d'] and values = [10,15,20,25], create a
dictionary including only pairs where value > 15.
```

#DECORATORS

It is a function which adds special/additional feature in the program.

There are 2 type:

1. built in decorator: predefined by developers.

```
i. @classmethod
ii. @staticmethod
iii. @abstractmethod
iv. @property
```

2. user defined decorator: created as per the requirement of the programmers.

#Syntax of user defined decorator:

```
def decorator_name(func): #func stores the address of the main func
    def inner(*args,**kwargs): #packing values
        #pretask
        func(*args,*kwargs) #main func calling ; unpacking values
        #posttask
    return inner
```

```
@decorator_name
def f_name(args):
    #SB
    f_name(parameter)
```

#A--B--C

```
print('visit website www.instagram.com')#pre task
print('login to account')#pre task
print('post a photo') #main task
print('logout from the account')#post task
```

print()

```
print('visit website www.instagram.com')#pre task
print('login to account') #pre task
print('like the post') #main task
print('logout from the account')#post task
```

print()

```
print('visit website www.instagram.com')#pre task
print('login to account')#pre task
print('comment on the post') #main task
print('logout from the account')#post task
```

```
def instagram(func):
    def inner(*a,**b):
        print('visit website www.instagram.com')#pre task
        print('login to account')

        func(*a,*b) #main func calling

        print('logout from the account') #post task
    return inner
```

```
@instagram
def A():
    print('post a photo')
A()
```

print()

```
@instagram
def B():
    print('like the photo')
B()
```

print()

```
@instagram
def C():
    print('comment on the photo')
C()
```

#Create a decorator to find the total execution time of a program.

import time

```
def timer(func):
    def inner(*a,**b):
        start_timer=time.time() #pre task
        func(*a,*b) #main func calling
        end_timer=time.time() #post task
        print("Total execution: ",end_timer-start_timer)#post task

    return inner
```

```
@timer
def add(n,m):
    print('Sum:',n+m)
add(45,45)
```

```
@timer
def rev(n):
    print('Reversed string:',n[::-1])
rev('data')

#Create a decorator which will provide 5 secs delay of program before execution.

import time

def delay(func):
    def inner(*a,**b):
        time.sleep(7) #pre task
        func(*a,*b)
    return inner

@delay
def multiply(a,b):
    print('Product of: ',a*b)
multiply(8,9)
```

```
""">#Practice Questions on decorators:
(29-08-2026)
```

1. WAP to create a decorator which should always return positive number.
**abs() to absorb the negative value to give positive nos.
2. Create a decorator to wish the employees of a company Happy Holiday.

The message should be like this:

Hi [Employee name],

[Write a message]

Thank you,
HR of [Company name]

Example:
Hi Ishani,
Happy Holiday,enjoy your weekend.
Thank you,
HR of Qspiders.

3. Create a decorator that counts how many times a function has been called.
4. Create a decorator that prints the function name before executing it.
5. Create a decorator that prints "Access Granted" before executing a function.
6. Create a decorator that checks whether a number is even before executing a function.
7. Create a decorator that measures and prints the execution time of a function.

#Exception Handling

exceptions are the errors in the program.

There are 2 types:

1. built-in exceptions
2. userdefined exceptions

#how to handle the exceptions?

#to handle exceptions there are 4 blocks:

```
try:
    #code with the error
except Exception_name:
    #soltuion of the error
else: #optional
    #this block will execute if there is no excpetion/error
finally:
    #this part gets executed everytime.
```

```
try:
    a=int(input('Enter a:'))
    b=int(input('Enter b: '))
    div=a/b
    print(div)
except (ZeroDivisionError,ValueError) as d:
    print(d)
else:
    for i in range(1,11):
        print(i,end=',')
finally:
    print('\nhello world')
```

#User defined exception:

#Steps:

#create a exception class from the base class i.e: Exception.
#use the keyword 'raise' to raise the exception.
#give the exception a solution.

#Create an exception where a integer no cannot be divided with 6.

```
class SixDivisionError(Exception):
    pass
```

```
try:
    n1=int(input('Enter integer no:'))
    n2=int(input('Enter integer no:'))
    if n2==6:
        raise SixDivisionError
    div=n1/n2
    print(div)

except SixDivisionError:
    print('division by 6 is not possible.')

except ValueError as e:
    print(e)
```

```
""">#practice question
31-08-2026
```

1. Age Validation

Create a user-defined exception called InvalidAgeError.

Ask the user to enter their age.

If age is less than 18, raise InvalidAgeError.
Otherwise print "Eligible to vote".

2. Insufficient Balance

Create a custom exception called InsufficientBalanceError.

Given:

balance = 5000

Ask the user for a withdrawal amount.

If withdrawal amount > balance → raise InsufficientBalanceError
Otherwise deduct the amount and display the remaining balance.

#FILE HANDLING

what is file?

File is a named location on the disk to store information, it is used to store data permanently. Since data is stored in a non-volatile memory we can retrieve it whenever required.

There are 3 types:

text file
binary file
csv file

#Text File

Stores data in ASCII or Unicode format.

Each char occupies 1byte of memory space.

Eg: '123.45'--6 bytes

'abstraction'--- 11 bytes

It is readable and understandable by the users.

Each line is terminated by \n or \r

it have the extension of .txt

Steps of handle a file:

1. Open a file
2. Perform operation
3. Close the file

Modes: write(w), read(r), append(a)

#opening a file

```
open():  
file_handle=open('file_name.txt','mode')
```

#perform operations

Write(w)

1. write(): it is used to write single line of data.

```
file_handle.write('data/message')
```

2. writelines(): it is used to write multiple lines of data.

```
file_handle.writelines('messages')
```

#Read (r)

1. read(): it will read the whole file/context as a single data.

```
file_handle.read()
```

2. readline(): it will read one line at a time.

```
file_handle.readline()
```

3. readlines(): it reads the file as a list of string.

```
file_handle.readlines()
```

** Output/the context will be displayed in the consol.

#Append(a)

to add data in an already existing file.

** function used will be of write mode.

#close of the file

close(): it will save and close the file

```
file_handle.close()  
"""
```

#write in file

```
f=open('Bangalore.txt','w')  
f.write('The Electronic City- Bengaluru')
```

```
f.writelines('''\n Bengaluru (also called Bangalore) is the capital of India's southern Karnataka state. The center of India's high-tech industry, the city is also  
known for its parks and nightlife. By Cubbon Park, Vidhana Soudha is a Neo-Dravidian legislative building. Former royal residences include 19th-century Bangalore  
Palace, modeled after England's Windsor Castle, and Tipu Sultan's Summer Palace, an 18th-century teak structure. ''')
```

```
f.close()
```

#with:

```
with open('Delhi.txt','w') as x:  
    x.write('The Capital of India')  
    x.writelines('''\n\n Delhi, India's capital territory, is a massive metropolitan area in the country's north. In Old Delhi, a neighborhood dating to the 1600s,  
stands the imposing Mughal-era Red Fort, a symbol of India, and the sprawling Jama Masjid mosque, whose courtyard accommodates 25,000 people. Nearby is Chandni
```

```

Chowk, a vibrant bazaar filled with food carts, sweets shops and spice stalls.'')

#read the file

with open('Delhi.txt','r') as v:
    #print(v.read())
    #print(v.readline())
    #print(v.readline())
    #print(v.readline())

    print(v.readlines())

#Append

with open('Delhi.txt','a') as v:
    v.write('\n\nDelhi is very famous because it is the vibrant political capital of India and an ancient city filled with thousands of years of layered history')

with open(r'/content/Python Syllabus.txt','r') as x:
    print(x.read())

with open(r'/content/Python Syllabus.txt','a') as x:
    x.write('\n Final Mock')

""">#CSV File

CSV: comma seperated value file.

each and every value will be seperated by comma ( , )

CSV store table format data

CSV file can be imported/exported from a program easily.

the extension is .csv

Steps to handle a file:

1. Open a file
2. Perform operation
3. Close the file

Modes: write(w), read(r), append(a)

#Write(w)

1. writerow(): to write single row of data.

Syntax: file_handle.writerow([data])

2. writerows(): to write multiple lines of data.

Syntax: file_handle.writerows([data],[data2],...)

""">

#write csv file

import csv

with open('Student_details.csv','w') as d:
    w=csv.writer(d) #writer(): allows us to write in the file
    w.writerow(['Name','class','section','roll','marks'])

    w.writerows([[['Sneha',12,'A',1,67],
                    ['Rashmi',12,'A',2,78],
                    ['Puja',12,'A',3,89]])

#read a csv file

import csv

with open('Student_details.csv','r') as r:
    read=csv.reader(r) #reader will give us access to read the file
    for i in read:
        if i: #it will skip the empty lines
            print(i)

#append data in csv file

import csv

with open('Student_details.csv','a') as d:
    w=csv.writer(d) #writer(): allows us to write in the file
    w.writerow(['Ishani',12,'A',4,70])

""">#practice Questions (01-09-2026)

Exercise:
1. count char in a text file
2. count lines in a text file
3. count words in a text file
4. count alphabets,digits,spaces & special char in a file
5. count alphabets,digits,spaces & special char in each line of a file
6. count vowels & consonants in a text file
7. count vowels & consonants in each line of a file.
8.count words containing vowels in a file.

Exercise:
1. Copy data from file 'a' to file 'b'
2. Read file 'a' & write number of words in file 'b'
3. Read file 'a' & write it in file 'b' in Title format
4. Read file 'a' & write each word in a separate line in 'b'.
5. Copy data from file 'a' to file 'b' & add line numbers in file 'b'.
6. Copy data from file 'a' to file 'b' & capitalize each line
7. Copy data from file 'a' to 'b' after replacing alphabets with *, digits
with #, and spaces with _

#Binary File Handling

the files containing audio,video,images,pdf,etc.

this stores the data in th same format in the memory.

binary files donot need any translation.

in binary files we can only read/write.

extension of binary file is .dat

in binary file 'pickle' module is used.

dump(): to write and save data in the binary file.

load(): to read the data from the file.

```

the data in the binary file is always encrypted(protected).

```
Modes: write (wb)   read (rb)
"""
```

```
#write in binary file
```

```
import pickle
```

```
with open('Demo.dat','wb') as f:
    text=['Good mornig','what are you doing','i am working now']
    pickle.dump(text,f)
```

```
import pickle
```

```
with open('Demo.dat','rb') as f:
    l=pickle.load(f)
    print(l)
```

```
""">#Json file Handling
```

A Json file is used to store and exchange data in a structured format.

Json: JavaScript Object Notation

Why JSON is used?

1. light weight
2. easy to read
3. easy to write
4. human readable
5. supported by all programming language.

****Json is alternative of CSV file.**

```
.json is the extention
"""
```

```
import json
```

```
data={'name':'Ishani','city':'Kolkata','job':'trainer'}
with open('demo.json','w') as f:
    json.dump(data,f) #to write in json file
```

```
import json
```

```
with open('demo.json','r') as g:
    x=json.load(g)
    print(x)
```

```
""">#Regular Expression (RegEx)
```

It is aprocess where by writing an expression any match/pattern will be fetched.

Tokens:

[A-Z]: to match uppercase char
[a-z]: to match lowercase char
[0-9] or \d: to match digits
[A-Za-z0-9] or \w: to match alphanumeric char.

\: it is used to match any pattern with special char.

\# *

```
use the module 're'
"""
```

#search(): re.search(pattern,data) it will give us the output for the first matched pattern from the data.

```
import re
```

```
text='Python is a high-level programming language. Python is easy to read and write programming language. Python is a multi-paradigm programming language.'
```

```
pattern='python'
```

```
s=re.search(pattern,text,re.IGNORECASE)
```

```
print(s)
```

#match(): re.match(pattern,data) it will give us the very first matched pattern from the data.

```
import re
```

```
text='Python is a high-level programming language. Python is easy to read and write programming language. Python is a multi-paradigm programming language.'
```

```
pattern='python'
```

```
m=re.match(pattern,text,re.IGNORECASE)
print(m)
```

```
patternl='language'
n=re.match(patternl,text)
print(n)
```

#finditer(): re.finditer(pattern,data) it gives all the matched patterns from the data.

```
import re
```

```
text='Python is a high-level programming language. Python is easy to read and write programming language. Python is a multi-paradigm programming language.'
```

```
pattern='is'
```

```
iter=re.finditer(pattern,text)
for i in iter:
    print(i)
```

#findall(): re.findall(pattern,data) it will give all the matched patterns in the form of list.

```
import re
```

```
text='Python is a high-level 9 programming language. Python is easy to read and 9 write programming language.9 Python is a multi-paradigm programming 9 language.'
```

```
pattern='9'
```

```
find=re.findall(pattern,text)
print(find)
```

```
#Mobile number matching
```

```
#10 digits
```

```

#1st digit and all 9 digits

#[6-9]{1}[0-9]{9}

#[6-9]\d{9}

import re

text='Some of the important phone numbers are 9876543210, 8765432190,7765432199,2345678891'

pattern='[6-9]\d{9}'

find=re.findall(pattern,text)
print(find)

#Match dates

#DD/MM/YYYY---Indian
#YYYY/MM/DD---International

#\d{2}-\d{2}-\d{4}

text='Today is 02-09-2026. Yesterday on the day of 01-09-2026 i went out for a trip. I will go back home on 10-09-2026.'

pattern='\d{2}-\d{2}-\d{4}'

find=re.finditer(pattern,text)

for i in find:
    print(i)

#match gmail and print if it is valid. Eg: 'ishanibanerjee@gmail.om'---Valid  'ishanibanerjee@outlook.com'---Invalid

#match website and print if it is valid or not.

#eg: 'www.google.com'--valid
#eg: 'www.yahoo.in'--not valid

"""#Package Architecture

Module are the .py files which contains reusable codes such as functions, classes, variables or statements.

Eg: math,time,re,csv,pickle,abc,...

Package is a folder/directory which is used to organize Python modules.

Eg: numpy,pandas, matplotlib,...
"""

import math
print(math.factorial(9))

```